

University of Greenwich
Faculty of Engineering and Science
School of Computing and Mathematical Sciences

Retrieving objects from short videos using text descriptions

https://github.com/mattvall1/fyp-video_object_timestamp_finder

Matthew Vallance
000000000

Supervisor: Tuan Nguyen

Word count: 12933

COMP1682 Final Year Individual Project

*A dissertation submitted in partial fulfilment of the requirements for the degree of
BSc (Hons) Computer Science*

May 7, 2025

Abstract

This dissertation explores the retrieval of objects from short video clips using text descriptions generated via image captioning techniques. Recent advancements in these areas and models such as CLIP, BLIP and Florence2 successfully generate accurate and reliable captions for static images. The motivation behind this project is to apply these techniques to video object detection, as this remains largely unexplored.

The main research objectives focus on developing an easy-to-use software package capable of identifying the exact timestamp where a specified object is visible within a video. This involves analysing candidate image captioning algorithms, overcoming computational limitations, implementing language processing and providing precise timestamps for found objects.

The methodology used to achieve this objective begins with a thorough literature analysis of existing techniques, followed by a brief user interface prototype. Next, computational challenges are addressed using key framing, with an exposition on a binary search theory to optimize performance on available hardware. Language processing tools are then employed to match frames to user-inputted search terms. These elements are integrated into a Python application that provides accurate timestamp retrieval based on a user inputted video and an accompanying search term.

Key findings reveal that despite some drawbacks, integrating image captioning algorithms and frame-selection techniques effectively enabled precise object detection and timestamp retrieval within videos. While computational limitations posed challenges in certain scenarios, this project successfully demonstrates the amalgamation of these techniques on consumer-grade hardware. The user-friendly interface showcases ease-of-use alongside robust functionality, paving the way for future improvements and broader applications in the field.

Acknowledgements

Throughout the course of this project, to which I have dedicated significant time and effort, there are a few notable individuals whom I would like to acknowledge.

I'd like to thank Tuan Nguyen, my supervisor for this dissertation. At the start of the academic year, Tuan provided key papers which helped form the idea, and overall aim of the project. In addition to this, Tuan maintained regular contact through weekly meetings in which he guided me in my development process and continually gave me genuinely useful feedback on how to improve my codebase. He was always available to answer my questions via Microsoft Teams, throughout my journey from an initial idea to bringing this project to life.

In addition, I'd like to thank the family members who proofread this report in its entirety. Despite not being immersed in Computer Science, I appreciate their patience and dedication in proofreading my work.

This degree overall has challenged me both academically and personally and I am proud to demonstrate everything I have learned during my time at the University of Greenwich. I hope this project proves as insightful to readers as it was rewarding to create.

Matthew Vallance
Greenwich, London, United Kingdom, May 2025

Table of Contents

Abstract.....	ii
Acknowledgements	iii
List of Tables	viii
List of Figures	ix
1 Introduction	1
1.1 Background	1
1.2 Aims and objectives	1
1.3 Methodology.....	1
1.4 Report roadmap.....	3
2 Literature Review	4
2.1 Introduction and Approach.....	4
2.2 Image captioning algorithms.....	5
2.2.1 General overview.....	5
2.2.2 Types of learning	5
2.2.3 CLIP (OpenCLIP).....	6
2.2.4 Florence2.....	6
2.2.5 RegionCLIP	7
2.2.6 VLP.....	7
2.2.7 BLIP.....	8
2.2.8 YOLO.....	8
2.3 Frame selection	9
2.3.1 Key framing	9
2.3.2 Binary Search Theory.....	10
2.4 Datasets.....	11
2.5 Automated testing metrics.....	12
2.6 Language processing.....	13
2.7 Conclusion	14
3 Product Research	15
3.1 Introduction	15
3.2 APIs.....	15
3.3 General purpose products	15
3.4 Conclusion	16

4	Requirements Analysis.....	17
4.1	Summary.....	17
4.2	Functional requirements listing	17
4.2.1	<i>Must (have)</i>	17
4.2.2	<i>Should (have)</i>	18
4.2.3	<i>Could (have)</i>	18
4.2.4	<i>Won't (have)</i>	18
4.3	Non-functional requirements listing	18
4.3.1	<i>Must (have)</i>	18
4.3.2	<i>Should (have)</i>	18
4.3.3	<i>Could (have)</i>	18
4.3.4	<i>Won't (have)</i>	19
4.4	Constraints	19
5	Design.....	20
5.1	Introduction	20
5.2	User interface.....	20
5.3	Processing and classes	21
5.4	Conclusion	23
6	Implementation.....	24
6.1	Review of technology	24
6.2	Prototype/GUI.....	25
6.3	Data retrieval.....	26
6.4	Image captioning analysis	26
6.4.1	<i>Automated Script Results</i>	27
6.4.2	<i>Manual Results</i>	28
6.4.3	<i>Concluding statement</i>	30
6.5	Frame selection	30
6.5.1	<i>Binary Search Theory</i>	30
6.5.2	<i>Key framing algorithm</i>	31
6.6	Language handling.....	32
6.7	Frame number retrieval	33
6.8	Summary.....	33
7	Results.....	34
7.1	Key results	34
7.2	Unit testing	35

7.3	Manual testing	36
7.4	Feature Demonstration	37
7.4.1	Output.....	41
7.5	Summary.....	42
8	Evaluation.....	43
8.1	Outcomes	43
8.2	Limitations	43
8.3	Unexpected challenges	44
8.3.1	Poor performance	44
8.3.2	Automated testing.....	44
8.3.3	Unit testing	45
8.4	Summary.....	45
9	Legal, Social and Professional Issues.....	46
10	Conclusion and Future Work.....	47
10.1	Conclusion	47
10.2	Future work.....	48
10.3	Closing remarks.....	48
11	References.....	49
	Appendix A – Project proposal.....	54
A.1	Introduction	54
A.2	Problem Domain.....	55
A.3	Methodology.....	57
A.4	Project Development	58
A.4.1	Prerequisites.....	58
A.4.2	Training and analysis of captioning algorithms	58
A.4.3	Reliably find objects within static images.....	59
A.4.4	Find ‘keyframes’ and apply object detection.....	60
A.4.5	Searching frames theory and implementation.....	60
A.5	Implementation	61
A.5.1	Tools and Technologies	61
A.5.2	Graphical User Interface.....	61
A.6	Project Plan	62
A.7	Evaluation	63
	Appendix B – Extended Results.....	65

B.1	Image captioning	65
<i>B.1.1</i>	<i>Automated script (third-party data)</i>	<i>65</i>
<i>B.1.2</i>	<i>Manual script (curated data)</i>	<i>74</i>
B.2	Unit testing	78
B.3	Manual testing	80
<i>B.3.1</i>	<i>Feature testing</i>	<i>80</i>
<i>B.3.2</i>	<i>Performance testing.....</i>	<i>81</i>
B.4	PDF report output	84
<i>B.4.1</i>	<i>Turtle sample</i>	<i>84</i>
<i>B.4.2</i>	<i>Mixed scenes sample.....</i>	<i>86</i>
B.5	RAM usage	87
Appendix C – Datasets, Videos and Images.....		90
C.1	Manually collected testing images.....	90
C.2	Manually collected testing videos.....	93
C.3	Sample of TextCaps set	95
Appendix D – Declaration of AI use		97

List of Tables

Table 1: Methodology breakdown	2
Table 2: Testing statistics.....	27
Table 3: Results from automated model testing	27
Table 4: Summary of manual model testing.....	29
Table 5: Caption examples in manual model testing	30
Table 6: Feature listing ('F/NF' = 'Functional/Non-functional', 'I' = 'Implemented')	34
Table 7: Averaged unit testing results	35
Table 8: Manual testing overview	36
Table 9: Averaged completion time by video	36
Table 10: RAM usage	44

List of Figures

Figure 1: Project lifecycle flowchart	2
Figure 2: Gantt chart	3
Figure 3: Visual example of captioning model	5
Figure 4: Binary Search Theory – Theoretical example	11
Figure 5: Initial user interface design	20
Figure 6: Prototype User Interface	21
Figure 7: UML - Use case diagram	21
Figure 8: UML - Package diagram.....	22
Figure 9: UML - Class diagram.....	22
Figure 10: Visual captioning overview	27
Figure 11: Grouped bar chart to visualise results from automated testing.....	28
Figure 12: Avg. Processing time and caption length by model	29
Figure 13: Binary Search Theory - Problem example	31
Figure 14: Key frame histogram with greyscale frames (top), and histograms (bottom).....	32
Figure 15: Demo - Initial User Interface	37
Figure 16: Demo - Select File	38
Figure 17: Demo - Search term entry	38
Figure 18: Demo - Key framing process	39
Figure 19: Demo - Found matching frame	39
Figure 20: Demo - Report modal	40
Figure 21: Demo - Select report directory	40
Figure 22: Demo - Confirm report directory.....	41
Figure 23: Demo - Completion report PDF output	41

1 Introduction

1.1 Background

Image captioning is a widely researched area, with techniques such as BLIP, CLIP, and Florence2 demonstrating remarkable progress in generating captions for static images. Similarly, key framing methods have provided a solid framework for identifying significant frames within videos, though it seems not many have directly applied these techniques when investigating video object detection.

The motivation behind this project is that, despite these advancements, there doesn't seem to be a fully-fledged product where these features are implemented into a standalone, user-friendly solution, allowing the use of these technologies on widely available hardware designed for home use.

1.2 Aims and objectives

This project aims to harness the existing 'image-to-text' workflow to create an easy-to-use software package for finding the exact timestamp at which an object is visible on the screen within a video.

To achieve this, several key objectives built upon the proposal (see Appendix A.1) need to be brought together. Firstly, some candidate image captioning techniques must be analysed, implementing one of these into a sensible graphical user interface.

Next, computational constraints will need to be overcome, to allow for consumer-grade hardware to be used.

Then, Language processing must be implemented to apply the search term to a captioned frame, addressing any language ambiguities, such as synonyms.

Finally, the exact frame and timestamp at which an object is found must be meaningfully returned to the user.

1.3 Methodology

Before any concrete implementation plans were created, a general plan of the project was devised using a flowchart. This demonstrated the general methodology of the project and formed the basis of how the project would be implemented.

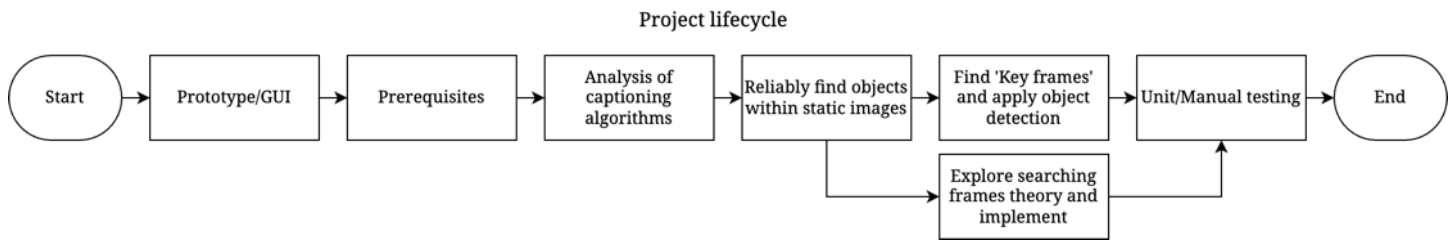


Figure 1: Project lifecycle flowchart

Table 1 shows a breakdown of the tasks required to complete this project, based upon Figure 1, though it does include additional steps for greater detail. This is not exhaustive, as each step will inevitably contain many sub-tasks.

Table 1: Methodology breakdown

Task name	Description
Complete planning	A thorough requirements analysis, including MoSCoW analysis should be completed
Finalize GUI	As part of the prototyping phase, the bulk of a GUI should have been created. Any remaining GUI additions should be created here. After this, the GUI should be complete, other than minor adjustments and placeholders being removed as the implementation commences.
Dataset, image and video acquisition	Find a dataset to analyse algorithms and compile an appropriate set of testing images and videos based on the project goal.
Algorithm analysis including finding objects in static images	This step involves two types of analysis: a longer format automated test using an external dataset (downloaded in the previous step) and manual testing using a choice selection of images (acquired in the previous step). Accuracy and quality are likely the most important metrics for this project over speed of captioning.
Key framing and binary search theory investigation	Explore keyframing methods and binary search algorithm theory as discussed in the methodology section. This is to overcome computational limitations when using consumer grade hardware to run the product.
Language handling	A search feature must be implemented, meaning that large amounts of data from the caption will need to be processed and/or removed. Along with finding relevant language features such as synonyms to speed up the search process.
Frame number retrieval	One of the project's final steps will be to find a method of retrieving the exact timestamp of the frame in which an object appears, effectively combining the language handling with the captioning algorithm.
Full test	Thoroughly test every aspect of the application, using a combination of manual and automated testing

Next, a Gantt chart was created to layout a rough timescale of the project week by week. By this point a prototype will have been implemented, each step can be cross referenced with Table 1.

Task	Week 1 20/01/25	Week 2 27/01/25	Week 3 03/02/25	Week 4 10/02/25	Week 5 17/02/25	Week 6 24/02/25	Week 7 03/03/25	Week 8 10/03/25	Week 9 17/03/25	Week 10 24/03/25	Week 11 31/03/25	Week 12 07/04/25	Week 13 14/04/25
Finalize GUI													
Dataset, video and image acquisition													
Algorithm analysis													
Single-image captioning													
Key framing and binary search theory													
Language handling													
Frame number retrieval													
Implement unit tests													
Full test (manual and unit)													

Figure 2: Gantt chart

1.4 Report roadmap

This report guides the reader through the project's research, design, and implementation of retrieving objects from short videos using text descriptions. Each chapter builds upon the previous one, providing a comprehensive project overview.

Firstly, the literature review offers a background on existing research in image captioning, key framing, binary searching within video frames, datasets for testing and accuracy metrics. The product research chapter then explores available solutions and concludes that most are not user-friendly.

The requirements analysis introduces the MoSCoW method of prioritization, functional and non-functional requirements along with project constraints. The design chapter then follows with the user interface design, processing routines, and internal classes using UML diagrams.

The implementation chapter provides an in-depth overview of the project, including technology used, prototyping phase, data retrieval steps, image captioning analysis, frame selection process, language processing and bug fixes.

The results and evaluation chapters examine the implementation with unit testing, manual testing results and feature demonstrations, along with the project's strengths and limitations. Legal, social and professional issues are then discussed, focusing on copyright, model discrimination and professional standards. Finally, the concluding chapter wraps up the project with an exposition on future work and ends with the closing remarks.

2 Literature Review

2.1 Introduction and Approach

A literature review has been conducted, during and after the project proposal was completed. This involved several key steps to ensure a thorough, and comprehensive review of existing research thus building an extensive working knowledge of the subject matter. This is essential in completing the project to a high standard.

Initially, some general papers on image captioning methods were provided by the project supervisor (included within this literature review) to develop general ideas around this project and the concepts of object detection in video. Analysis of these papers gave a good idea of further literature and concepts needing to be researched in depth. This led to a deep dive into different image captioning algorithms and techniques, such as transfer learning and zero-shot learning. Next, three image captioning candidates for implementing into this project are revealed and upon completion of the research, a method of applying these candidates to a video needed to be found. This is where research on key framing came into play. After image captioning and key framing had thoroughly been researched, some suitable datasets needed to be sourced, to allow for thorough testing of image captioning algorithms later in the project lifecycle. During the research of the project, the problem of word processing became apparent, leading to a research section on this.

Papers have mainly been sourced from IEEE Xplore during this research as it proved to be the most valuable resource for material on this subject. It provided excellent, in-depth academic articles on all subjects mentioned previously. The ACM digital library has also been used to provide academic research articles. Additionally, Google Scholar was used, as it returns articles, books and conference papers from a variety of academic publishers, online repositories, professional societies and universities.

While academic databases have proven to be extremely valuable, some aspects are not available, leading to using general web searches via Microsoft Bing. For example, where specific developer documentation of products was needed, along with opinions from internet resources such as periodical publications. Another excellent source of information has been Microsoft Copilot (see Appendix D) search terms such as ‘find an academic source which explains what WordNet is’ returned some genuinely valuable results. Sometimes these are locked behind paywalls, or specific memberships. However, if the word ‘open access’ is specified, it provides full-text sources. However, these queries do not always return valuable results, but each time this method was used, the paper was read offline to verify that it contained accurate, relevant and useful information.

In addition to this, some academic books have been sourced using open repositories, which have been helpful to gain knowledge of software development practices, rather than for literature review purposes.

2.2 Image captioning algorithms

2.2.1 General overview

An image captioning model generally consists of two main parts: the visual detector (image feature extractor) and the text generator. The image feature extractor is responsible for processing the input image and extracting relevant objects like a car along with shapes, colours, and sizes. A Convolutional Neural Network (CNN) is usually used for this, and it produces context vectors, which are detailed summaries of visual elements within the image. The text generator then takes the content of an image (produced by the feature extractor) and translates it into a coherent, descriptive sentence (Sharma & Padha, 2023).

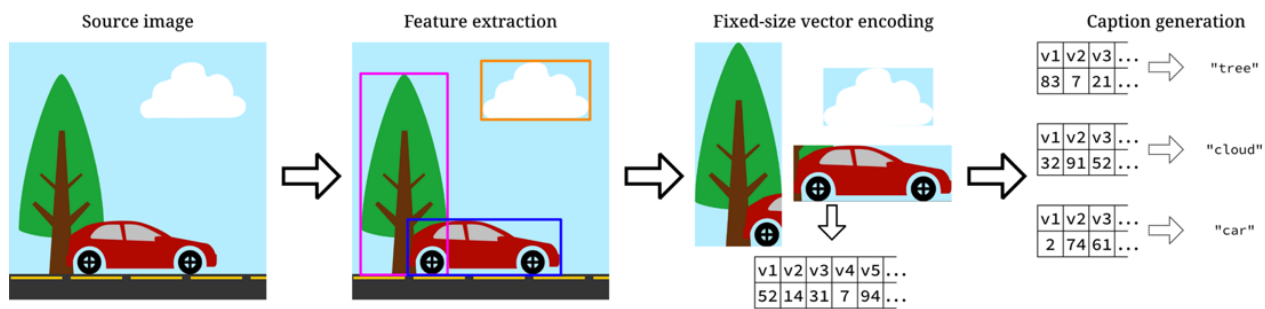


Figure 3: Visual example of captioning model

2.2.2 Types of learning

The following algorithms allow for zero-shot (ZSL), which can form the feature extractor's basis. Zero-shot learning is a method to recognise and classify objects the algorithm hasn't seen before, it is completed using side information such as attributes. During training, a ZSL model will collect semantic information, which involves extracting features from images and text, such as shapes, colours, words and phrases. Next, it maps the extracted features to a set of attributes, which in the context of ZSL are 'shared and nameable visual properties of objects' (Xian, et al., 2019, p. 2253); these are human-interpretable, for example: 'is red' or 'has legs'. Using seen classes (the predefined categories the model is trained on), the model can then learn the relationships extracted, features extracted and the corresponding attributes. If the model is presented with an unseen class, it can use the learned mappings to predict the attributes from the features of the new data. Finally, the model will classify the unseen classes. This is done by using the same feature extraction method used initially and then matching these features with the semantic information collected during training (Xian, et al., 2019).

Transfer learning (TL), conversely, is a method to recognise and classify objects that the algorithm has seen before. A TL model works on the principal that 'a person who has learned the violin can learn the piano faster than others since both the violin and the piano are musical instruments and may share some common knowledge' (Zhuang, et al., 2021, p. 44). Firstly, a pre-trained model needs to be selected, which has been trained on a large, pre-labelled dataset; this means that the pre-trained model will have already learned a vast number of features from an extensive dataset. This pre-trained model is then used as a feature extractor, processing new images and extracting their essential features. Layers can then be added on top of the pre-trained model, allowing the model to recognise specific objects related to the task given. These layers are then used to train the model further and will enable it to adapt to needs. The process of transfer learning effectively uses the pre-trained model to improve performance on the new task (Zhuang, et al., 2021).

2.2.3 CLIP (OpenCLIP)

A common approach to captioning tasks that has been thoroughly researched is Contrastive Language-Image Pretraining (CLIP). CLIP is a neural network trained on text/image pairs, enabling it to predict the most relevant text snippet for any given image. An essential part of natural language is that it can describe an enormous set of concepts; the CLIP model is trained with this in mind and, therefore, can associate text with images. Existing work uses readily available smaller datasets with around 100,000 pairs, and a new dataset has been constructed using publicly available sources on the internet for CLIP with 400 million image-text pairs. CLIP does not require a specific training stage, meaning it can perform Zero-Shot classification. The names of all classes are used as a set of potential text pairings, which allows it to predict the most probable text-image pair and produce a result based on text descriptions (Radford, et al., 2021).

OpenCLIP builds upon this original implementation. Notably, it was trained on the LAION-2B dataset, which contains 2-billion image-text pairs, this yields substantial performance gains in zero-shot learning tasks (Cherti, et al., 2023). Due to its open-source nature, it has been continually maintained via GitHub since its initial launch, meaning it is simple to implement (Wightman, et al., 2025).

2.2.4 Florence2

Florence2 is a computer vision model designed to handle a vast range of vision/vision-language tasks such as object detection, captioning, and grounding (linking text descriptions to specific regions in an image). Florence2 is capable of zero-shot learning and was trained using 5.4B annotations on 126M images, making it a versatile vision foundation model (Xiao, et al., 2024).

Florence2 generates captions using a sequence-to-sequence (seq2seq) architecture, integrating an image encoder along with a multi-modal encoder/decoder. Firstly, the input image is converted into visual token embeddings (using a vision encoder), which represent the images features in a format suitable for processing. Next, a text-based prompt is provided to specify the task (such as: '<CAPTION>', for captioning an image), which is converted into text embeddings using a tokenizer to break down the text into smaller units. Both the visual and text token embeddings are then combined and fed into a multi-modal (encoder/decoder) transformer which allows the model to understand this information. Finally, the decoder generates a description of the image based on the combined visual/text embeddings, which is then optimized using a cross-entropy loss function (this measures the difference between the predicted probability distribution and the original labels), which ensures accuracy and coherence (Xiao, et al., 2024).

Florence2 can generate several different caption lengths, those being 'Brief Caption', 'Detailed Caption' and 'More Detailed Caption'. The 'Brief Caption' is the fastest and it returns a single sentence, representing the most prominent objects and activities. Alternatively, using 'Detailed Caption' and 'More Detailed Caption' both return several richer sentences ranging in length. Of course, the longer the caption, the slower it generates but the more detailed the resulting text is (Xiao, et al., 2024).

2.2.5 RegionCLIP

RegionCLIP builds upon the original CLIP model discussed above, though it can caption specific regions instead of just captioning an entire image, demonstrating a 'major domain shift' (Zhong, et al., 2022, p. 16772), in which the original CLIP algorithm was trained to caption an entire image. RegionCLIP is a significant extension of this functionality as it focuses on specific parts of the image rather than just understanding the image as a whole; this is known as fine-grained alignment. It creates "pseudo" labels for each smaller region, which are then combined with ground-truth text-image pairs (the correct caption for the image). The model is then trained on this, improving its ability to detect and describe objects. In essence, RegionCLIP enhances the original CLIP model by enabling it to zoom in and understand the finer details within an image, making it better for detailed image analysis (Zhong, et al., 2022).

2.2.6 VLP

Next, the unified Vision-Language Pre-training (VLP) model. VLP is pre-trained on the Conceptual Captions dataset (Sharma, et al., 2018) consisting of 3 million text-image pairs. VLP is trained using two unsupervised learning objectives: Bidirectional Encoder Representations from Transformers (BERT) and sequence-to-sequence (seq2seq). BERT is a language representation model (human language generator) designed to pre-train by

conditioning on the left and right context (Devlin, et al., 2019); this allows it to understand the context of a word based on all the words around it rather than just the words before it. Seq2seq involves using a specific self-attention mask which controls the context available to the language model during prediction. VLP offers vast improvements in training speed and accuracy compared to older models due to its unified pre-training rather than traditional random or language-only pre-training (Zhou, et al., 2020).

2.2.7 BLIP

Finally, the Bootstrapping Language-Image Pre-training (BLIP) model. BLIP is a framework designed to enhance the VLP model discussed above, implementing several improvements such as a Multimodal mixture of Encoder-Decoder (MED), and the CapFilt method (Li, et al., 2022).

The MED is the core architecture of BLIP, and it handles both vision-language understanding, and text generation tasks. It unifies three functionalities: the processing of images and text, image-grounded text encoding which integrates visual and textual data, and an image-grounded text decoding for generating text from a visual input (Li, et al., 2022).

The CapFilt method supplements the MED by bootstrapping (progressing without external input) noisy web captions for improved dataset quality. It includes a captioner which generates synthetic captions for web images, and a filter which removes noisy (inaccurate, incomplete, or irrelevant) captions. By doing this, BLIP improves its pre-training process, equating to better performance in vision-language tasks, such as image captioning (Li, et al., 2022).

2.2.8 YOLO

You Only Look Once (YOLO) is a real-time object detection algorithm that treats object detection as a regression problem, which is applied to predict the bounding box coordinates for objects in an image. It works by first taking an input image and dividing it into a grid, each cell of this is then responsible for detecting objects within its boundaries by predicting bounding boxes within them and giving them confidence scores, which represent the likelihood of an object being present in that box. Next, a single CNN processes the entire image to simultaneously predict bounding boxes and class probabilities (which represent the likelihood of specific object classes being present). The confidence calculation then multiplies the class probabilities and confidence scores to produce class-specific scores for each bounding box. Duplicates are then eliminated, and the most confident predictions are kept, which are then outputted as bounding-box coordinates, confidence scores, and class probabilities (which are effectively the detected objects) (Redmon, et al., 2016).

While YOLO's feature set is extensive, its relevance to this project is limited due to it being an object detection algorithm, rather than an image captioning algorithm. However, YOLOv10 contains an object detection method returning an array of objects found (Wang, et al., 2024), which may be worth exploring.

2.3 Frame selection

2.3.1 Key framing

(Kaur, et al., 2024) proposes a key frame extraction method which addresses data redundancy in videos and improves video summarization. This explains that there are two types of key frame extraction video summarization algorithms: static and dynamic summaries. Dynamic summaries are a short video summary of a movie whereas a static summary is simply a group of summaries which don't change over time, such as images or text. It also discusses the most popular technique for key frame extraction, which works by looking at the difference between consecutive frames. Each frame whose differences are more than a certain threshold are selected as the key frames – features such as colour histograms, correlation and edges are used to calculate these differences. The clustering method clusters frames together according to some similarity metrics, and then key frames are selected from each of these clusters (Kaur, et al., 2024).

Another interesting method of key frame extraction is using a Cascaded MapReduce algorithm. Hadoop is used to store a large amount of video frames in the HDFS file system, then cascaded MapReduce blocks compute the colour histogram for each frame and the difference between consecutive frames using the Euclidian distance. A threshold has been set by calculating mean and standard deviations and each of the key frames is extracted by comparing the histogram difference with the preset threshold value. This process is sped up by using parallel processing, the larger the number of nodes, the faster the application is (Deshmane, 2017).

A particularly promising method of key framing is using the threshold technique, which is based on the absolute histogram difference between frames. This is broken up into three stages (known as modules). In the first module, all frames are taken from the inputted video and then stored. In the second module, two consecutive images are taken and converted to greyscale. Histograms are then created from these greyscale images, and the difference between these is calculated. In the third and final module, a threshold is calculated by using the following formula, where sd is the standard deviation of the differences, m is the mean of the absolute differences between every frame, and a is a constant value to raise or lower the resulting threshold (Ghatak, 2016):

$$t = sd + (m \times a)$$

This threshold (t) is then compared with the histogram difference values of each pair of consecutive frames, if the value is over the threshold, the frame is selected as a key frame. The larger the constant value (a), the higher the threshold will be; therefore the less frames will be selected, and vice-versa (Ghatak, 2016).

Based on these findings, several approaches could be utilized to retrieve the key frames in the video, enabling the efficient execution of a selected image captioning algorithm on them. These are the main two components of the project, with the frame selection aspect arguably being more important for the project overall and likely taking up a larger amount of time to develop compared to applying object detection algorithms to the images.

2.3.2 Binary Search Theory

A binary search is a ‘divide-and-conquer’ approach to searching for something in a sorted set of records by repeatedly halving the set until the target is found or deemed non-existent (Sedgewick, 1983). To retrieve relevant images, therefore reducing computational load, this simple theory of binary searching could be used, and should be explored as part of the implementation process. If a set of frames was taken from the initial maximum of 3600 frames, for example, 1 frame after every 60 frames have passed (or every 2 seconds of video), this leaves a maximum total of 60 images to apply the detection algorithm to. First, the algorithm would split the video up into the frames mentioned above and would look for the object. In Figure 4, a very simple video of a red ball rolling across the screen from left to right is used to demonstrate this. The first selected frame is blank, and as the algorithm skims through the video, it can see that the ball is fully visible within frame 120. The captioning algorithm would be able to understand this (for example, returning something like ‘red ball’ as opposed to the previous frame being ‘red semi-circle’), rewind to the halfway point between this frame and the last and move onto the 10th frame from there. This would be continued until the algorithm is scrubbing frame-by-frame.

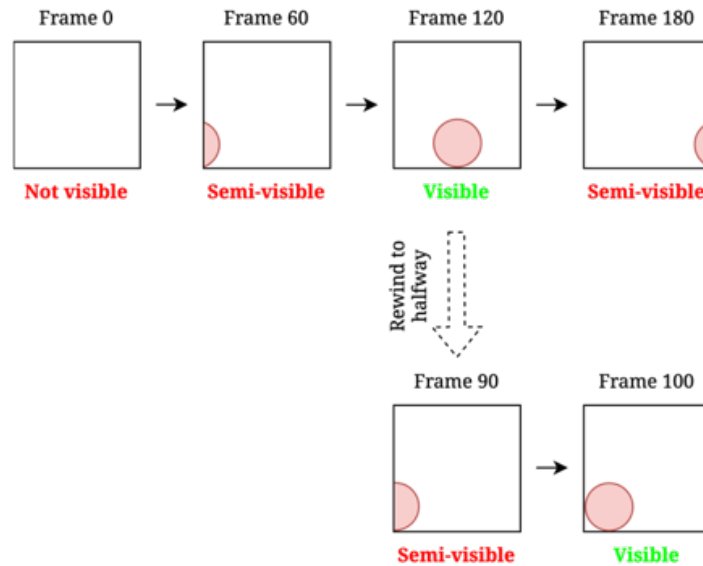


Figure 4: Binary Search Theory – Theoretical example

2.4 Datasets

To analyse the image captioning algorithms, another type of text-only dataset is needed, known as image caption annotations. There are several of these available with three notable options being Contextual Captions, TextCaps and nocaps all of which are openly available.

The Conceptual Captions dataset contains around 3.3 million image/description pairs. This data has been scraped from the web, using the alt-text associated with web images, which means it contains a wide variety of imaged and accompanying captions (Sharma, et al., 2018).

The TextCaps validation set contains around 145,000 captions for 28,000 images. These captions were created by humans using a two-step process. Annotators were first asked to describe an image in a single sentence, then evaluators were asked to vote on whether each caption written conformed to a set of requirements measuring the quality of each caption, leading to five captions being collected for each image (Sidorov, et al., 2020).

The nocaps dataset contains around 166,000 captions for 15,100 images from the Open Images V4 validation/test sets. To make the dataset more challenging, images with multiple objects and rare objects were selected, and images with single object classes were removed. The captions were collected using Amazon Mechanical Turk (a crowdsourcing marketplace, in which a pool of human intelligence is easily available), to receive eleven captions for each image (Agrawal, et al., 2019).

2.5 Automated testing metrics

It would be possible to evaluate the previously discussed captioning algorithms manually, however this is a lengthy process, and the scope of this project does not allow for the time required for full manual testing. This is where some widely used automatic evaluation metrics come in useful.

The following metrics share the same inputs: a caption to test (known as the candidate) and the reference captions, which are provided along with an image by a testing dataset.

Another common theme is the theory of ‘n-grams’, which are a sequence of ‘n’ items (in this case words), which are extracted from a given text, these sequences are used to predict the next word in a sentence. The 1-gram (unigram) precision verifies that accurate vocabulary is used. The 2-gram (bigram) precision measures fluency by requiring words to appear in correct pairs. The 3-gram (trigram) precision checks that short phrases are coherent and ordered correctly. Finally, the 4-gram verifies that longer phrases are coherent and ordered correctly (Kurdi, 2016).

Firstly, the Bilingual Evaluation Understudy (BLEU), which is a quick evaluation method that correlates well with human evaluation. BLEU works by measuring the overlap between the candidate and reference captions using n-grams. Initially, BLEU was designed to score translations, but more recently it’s been applied to image captioning among other machine learning tasks. BLEU first extracts the n-grams (1 thru 4) from the candidate and reference captions, and clipping is applied to make sure the count of n-grams doesn’t exceed their peak frequency in any reference. Using these n-grams, the modified precision is calculated as the number of clipped matches divided by the total number of n-grams in the candidate caption. A combined precision score is then calculated using the geometric mean of the modified precisions. Next, a brevity penalty is applied, which compares the length of the candidate with the closest matched (in length) reference caption and if the candidate is shorter than the selected reference, a penalty is applied. Finally, the BLEU score is calculated by multiplying the combined precision score with the brevity penalty to produce an output between 0 and 1, the closer to 1 the score is, the closer the candidate aligns with the human references (Papineni, et al., 2002).

Another metric is the Recall-Oriented Understudy for Gisting Evaluation (ROUGE) package which contains four measures: ROUGE-N (known as ROUGE-1/2 later), ROUGE-L, ROUGE-W and ROUGE-S. ROUGE-N works by comparing the overlap of n-grams between the candidate summary and reference summaries. It focuses on recall, meaning it measures how much of the reference is found in the candidate. ROUGE-L evaluates the largest common subsequence between the candidate and reference. This metric considers sentence structure and word order, making it useful for assessing fluency and coherence. ROUGE-W

builds on ROUGE-L by giving more weight to consecutive matches in the sequence, rewarding summaries that maintain the flow of ideas more effectively. ROUGE-S (skip-gram) measures the overlap of word pairs in their original order, even if they are not adjacent. This allows it to capture relationships between words while being less strict about their proximity (Lin, 2004).

Finally, the Metric for Evaluation of Translation with Explicit Ordering (METEOR), which aims to correlate closely to human judgements and is specifically designed to address weaknesses in the BLEU metric. METEOR focuses on unigram matches between candidate and reference captions, based on their exact forms, stemmed (base/root) forms, or their synonyms. It first identifies these unigrams, then among the best alignments (where every unigram in each candidate maps to 0 or 1 unigram in the reference), the one with the largest subset of matches is selected and if there are ties, the one with the fewest disordered word mappings is chosen. Then, the precision (ratio of matched unigrams to total unigrams in the candidate) and recall (ratio of matched unigrams to total unigrams in the reference) are calculated to get the fmean which weighs recall much higher than precision. All matching unigrams for each are then grouped into chunks and a penalty is applied based on the number of these – the higher the number of chunks, the higher the penalty is. Finally, the METEOR score is calculated as (Banerjee & Lavie, 2005):

$$\text{Score} = \text{fmean} \times (1 - \text{penalty})$$

2.6 Language processing

A key part of this project is the ability for a user to input their own search term and as the complexity of the text input can vary wildly, the program will need to have the ability to determine what it needs to find within a video. English has a wide array of ways to explain something, and the human mind can quickly understand these in many contexts when looking at an image. For example, to think about a 'red car' is a straightforward concept for a person, but this could be described in a seemingly infinite number of ways: 'Red car', 'car that is red', and 'red passenger vehicle' could all mean the same thing to a human, but a computer must be able to understand this in a similar way.

The English WordNet 2020 (now on version 2024) project is an open-source version of Princeton WordNet, which has only had one updated since the last release in 2006 (McCrae, et al., 2019). A WordNet is a large lexical database of a language. In a WordNet, each word is grouped into sets of synonyms known as synsets, each of these synsets represent a specific concept and include all words which can be used to express that concept. These are then connected via various semantic relationships, such as hypernymy/hyponymy (broader/narrower terms), antonymy (opposites) and meronymy (part-whole relationships, i.e. where one concept is considered part of another concept) (Miller, et al., 1990). WordNet

2024's extensive database of synonyms and semantic relationships allows it to understand that 'car' and 'motor' can be related terms. So, if a user describes a 'red car' as a 'red vehicle' WordNet can be used to help the captioning algorithm recognize that both terms refer to the same concept.

2.7 Conclusion

In this literature review, a deep dive into image captioning and methods to complete the project aims and objectives was carried out using a vast array of sources from academic journals, conferences and books to developer documentation.

The models discussed within this research can be used for zero-shot learning, allowing easy comparison within the project. This means the same datasets and training can be used, ensuring relevant and accurate testing results for algorithm selection. These models are relatively recent and represent a broad range of approaches and accuracy ratings. In addition, they have widely available codebases, making them efficient to apply and analyse. Three models selected for further investigation are OpenCLIP, BLIP, and Florence2 as they are pre-trained on large datasets and are open source. Florence2 is particularly interesting due to its ability to generate lengthy captions with detailed descriptions (Xiao, et al., 2024).

In addition, some key framing methods have been discussed, which allow for the removal of large amounts of useless data. This will speed up the entire workflow, particularly with the limited hardware resources available. This led to an exposition on datasets, where the importance of diversity became apparent, and the need for testing which model would be best to reach the objective became clear. The text-only datasets consisting of images and reference captions are vital for testing the three models selected for potential implementation.

Overall, the literature review has given a detailed background on image captioning models and their inner workings, methods of selecting the best frames for analysis, datasets for training and testing models along with automated metrics to analyse them. It ends with a discussion around language processing, paving the way for smooth planning and implementation of the final product.

3 Product Research

3.1 Introduction

This project aims to be a standalone product, integrating all aspects of a software to find the timestamp of an object in a video. There is a vast amount of research in this area, which means there are products on the market which can be used for the purpose of object detection in images and video. However, none of these seem to combine an image captioning algorithm, word processing and a sensible, easy-to-use User Interface (UI), which would allow less technologically capable individuals to use this software.

3.2 APIs

Google offers two different Cloud APIs for object detection. The Google Cloud Vision API offers object localization that detects and extracts multiple objects within an image, providing an annotation for each object, which includes information about the object, its position, and rectangular bounds. It can also identify significant and less-significant objects in an image. Additionally, it provides a confidence scores for each detected object (Google, 2025). In contrast, Google Cloud Vertex AI Vision offers advanced video object detection capabilities. It can detect and locate over 500 types of objects within a video, providing bounding boxes, object labels, and confidence scores for each video frame. The results also include a timestamp, allowing for tracking objects over time, but it will not stop once it detects the full object, which is an integral part of this project. Both Google API products return a string for each detected object in an image/frame, mirroring some of the researched captioning algorithms (Google, 2025).

Another similar product is Roboflow. It offers a comprehensive suite of tools for image object detection and captioning. In object detection, it supports various publicly available models such as YOLOv8, which (as discussed earlier) can detect and locate objects within images. Roboflow is another platform for developers to integrate these tools, rather than aiming at the public (Roboflow, 2025).

3.3 General purpose products

There are, of course, products like Google Photos and Apple Photos, both of which integrate object detection to automatically tag and organise photos. These are designed to be easy-to-use and effortlessly integrate object detection techniques into products used by the public. Within the Apple Photos app, users can search for objects within images. Apple Intelligence uses a natural language search feature to retrieve images related to a text prompt a user

enters, which is a step closer to what this project aims to be. However, this is based on a user's photo library, rather than searching within a specific image/video (Michaels, 2024).

3.4 Conclusion

As mentioned in the introduction of this section, this project is designed to be easy-to-use for the average person, none of the solutions discussed within this section achieve that goal, and generally they are just extensions, or implementations of already available technologies. Whilst these products are great for people with technical knowledge, they leave others behind, leaving a gap in the market for a solution like this project.

4 Requirements Analysis

4.1 Summary

The requirements analysis is the first stage of delivering the completed product. It is well known that project requirements are split into two types, functional and non-functional. Functional requirements describe what the product must do, its specific functionality and the tasks it must perform, in other words it outlines the actions and operations of the software. In comparison, non-functional requirements describe how the software should perform and how usable it is. “Functional requirements are things the product must do” (Robertson & Robertson, 2013, p. 10), and “non-functional requirements are qualities the product must have” (Robertson & Robertson, 2013, p. 10).

Another important feature of the requirements analysis is the constraints. Constraints are project-wide requirements; these are the limitations (Robertson & Robertson, 2013). In the context of this project, they form some prerequisites or rules which are inescapable.

A well-known and highly useful technique for prioritizing and analysing requirements called MoSCoW has been used for both functional, and non-functional requirements. MoSCoW splits requirements into four categories: Must have, should have, could have and won't have. ‘Must have’ means these requirements **must** be included, if any of these are not delivered, the project is considered a failure. ‘Should have’ reduces the importance – these requirements are considered **highly important** but are not critical for delivery. ‘Could have’ further reduces the importance, these are less critical and aren't necessary to implement but are **nice to have**. Finally, ‘won't have’ are requirements that have been considered but **are not implemented** (Cobb, 2015).

4.2 Functional requirements listing

4.2.1 Must (have)

- Split a video into individual frames
- An image captioning algorithm as the base of the project
- **One of the following** methods of reducing the number of images the image captioning algorithm needs to process:
 - Analysis of frames to retrieve the most relevant ones
 - An implementation of the binary search theory
- The ability to specify a video file
- The ability to specify a word to search within the specified video file
- Find synonyms for the user inputted word to search within captions
- Provide the first timestamp for a word within a video

4.2.2 Should (have)

- A processing length of no longer than 30 minutes
- A warning if the length of the video is too long (2 minutes, but still allow longer)
- The ability to handle videos of different formats and resolutions
- A highly accuracy image captioning method
- Exception handling
- Process logging

4.2.3 Could (have)

- The ability to pause execution
- A final report in PDF format, with all relevant findings
- A method of finding specific phrases within a video, not just a word (and its synonyms)
- An enforced limit on the length of the video importable to 2 minutes (or ~3600 frames)

4.2.4 Won't (have)

- Multiple captioning methods/iterations of the same method
- User-updatable parameters (such as number of iterations to perform)
- A database system of results using MySQL

4.3 Non-functional requirements listing

4.3.1 Must (have)

- A simple Graphical User Interface (GUI)
- Implementation in an Object-Oriented software paradigm
- File format limits so that only compatible video files can be used

4.3.2 Should (have)

- High reliability
- High accuracy
- Feedback on progress
 - Progress bar
 - Console/logging

4.3.3 Could (have)

- Feedback on progress
 - Frame counter
 - Total time elapsed

- Parallelisation – Process multiple images at the same time (could benefit performance)

4.3.4 Won't (have)

- A video queueing system

4.4 Constraints

There are few constraints on both the development and delivery of this project. Firstly, the time constraint. There is a set date by which the project must be completed, in whatever state it may be in. This immediately limits the scope of a project, meaning the project requirements are likely the most important part of delivering a solid product.

Another constraint is current solutions to image captioning as any proposed solution will be constrained currently available technology. As discussed within the Literature Review, no method of image captioning is totally accurate, meaning that any project solution is ultimately limited by existing technologies.

A final important note is that there are limits on the hardware available. The project has mainly been developed on an Apple MacBook Pro, with an Apple M1 Pro, therefore using macOS, which means it can't be guaranteed to work on other platforms without extensive testing. Some initial model testing will be performed using an Intel-based Microsoft Windows machine, with an Nvidia RTX 3060 GPU, which greatly increases model performance testing when looking at vast datasets but is not realistic for most people. While this is a constraint, it could also be seen as an advantage, as this project is meant to be for the public, there would be limits to their resources as well.

5 Design

5.1 Introduction

The design process has been split into two phases, first the user interface, then the logic. Due to this project being aimed at the public, ease-of-use is paramount. Figure 5 shows an early interface design (more details within Appendix A.5.2), which gave a rudimentary idea of how the project could look and therefore how it could function.

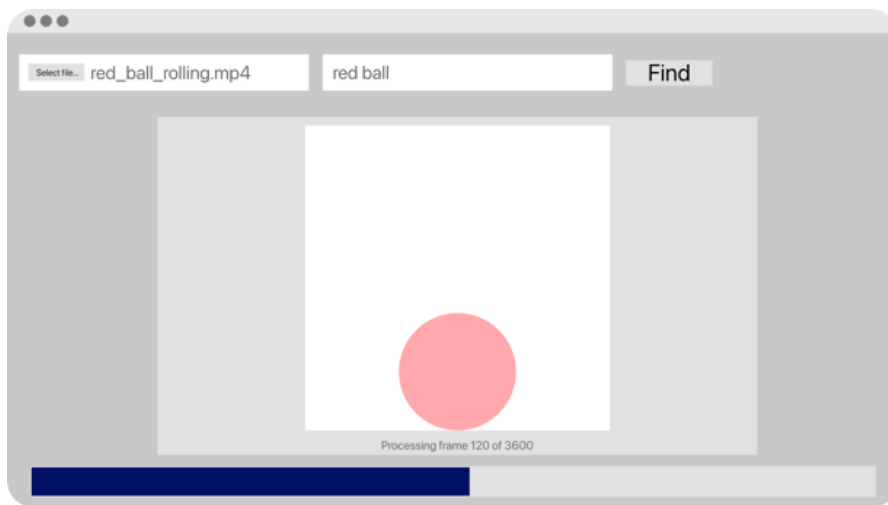


Figure 5: Initial user interface design

The software architecture design was aided by using Unified Modelling Language (UML) to create diagrams, which are powerful tools due to their ability to visually represent elements of a software system such as requirements, data structures, and information and data flows (Seidl, et al., 2015).

5.2 User interface

Amalgamating all these elements into one sensible Graphical User Interface (GUI) is challenging. Several GUI elements are needed, starting with a text input for the user to enter their prompt and then a file picker for selecting the video file in question. A preview window, which shows the current frame in which the software is currently processing would be beneficial so that the user can determine progress so far. An initial image of the design was created in graphic design software (see Figure 5), then was translated into the prototype. Figure 6 shows the initial working prototype with an image to give an idea of how the program would look and feel.

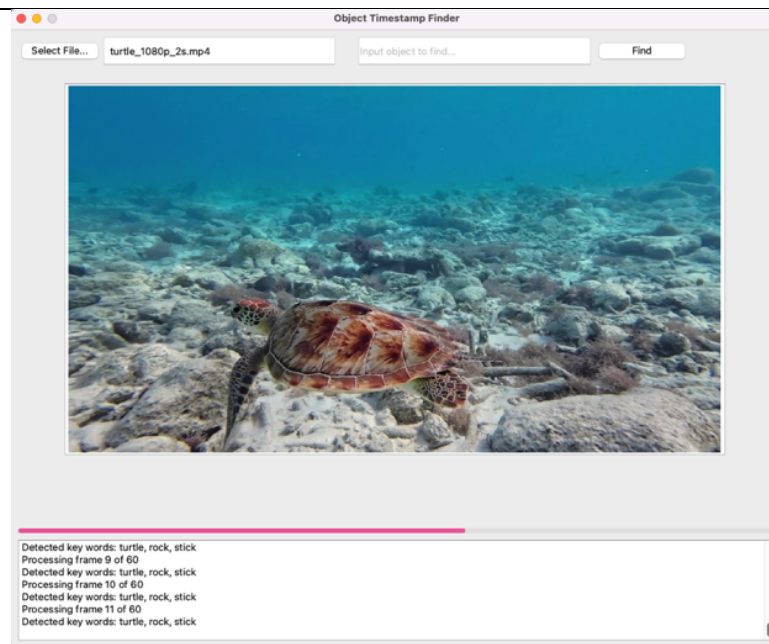


Figure 6: Prototype User Interface

The idea of this prototype was to get a general feel for the software. Each step of the processing is saved to the in-app console with a report generated at the end of the process allowing the user to leave the app unattended then return with the information that they wanted. If the user searched 'turtle', the frame, its accompanying caption and timestamp where a turtle appeared would be saved to the report.

5.3 Processing and classes

One of the most useful parts of OOP, is the separation of objects, this makes development highly maintainable and efficient. Unified Modelling Language (UML) visualisations were created to aid in implementation. Firstly, Figure 7 shows a simple use case diagram which was created to indicate the scenarios in which the user would interact with the project.

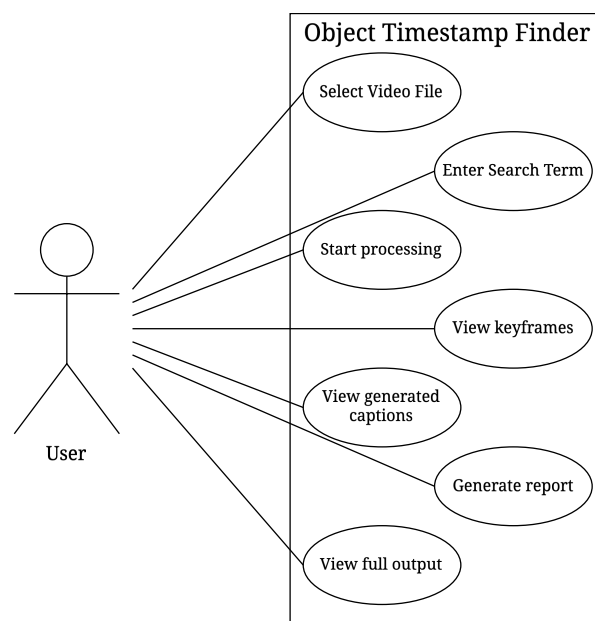


Figure 7: UML - Use case diagram

The second UML diagram to be created was a package diagram, which focuses on high-level organisation of the application. Packages allow you to group elements such as classes and data types (Seidl, et al., 2015). Figure 8 shows the project will be organized into three distinct packages within the overarching 'App': the user interface, the processing and a 'GlobalTools' class for any tools or routines which may be needed on startup and within both internal packages.

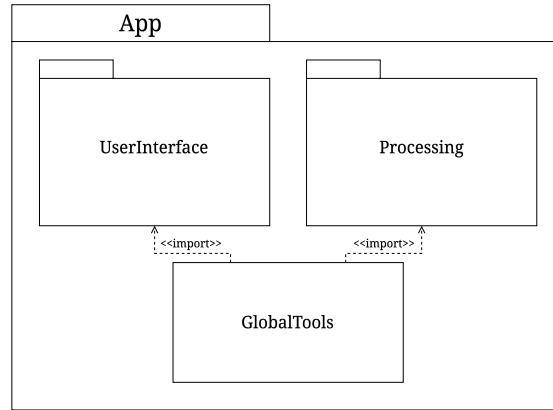


Figure 8: UML - Package diagram

To dive further into how the product would function, a class diagram was created. Class diagrams can be used to both plan elements and to document existing code (Seidl, et al., 2015). The diagram in Figure 9 shows the diagram after completion of the project, which has been updated over the course of development, initially it only consisted of titles and basic functionality.

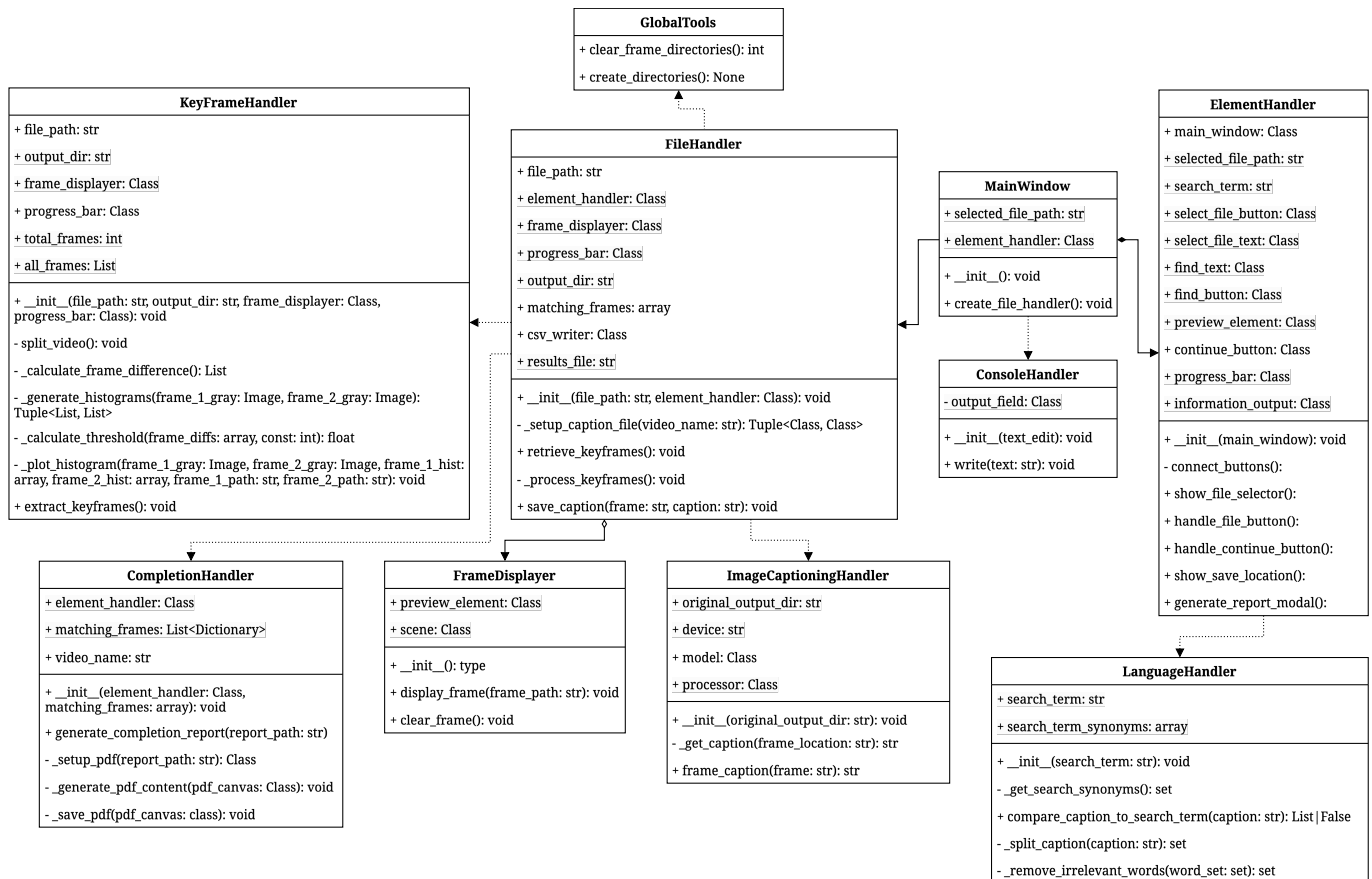


Figure 9: UML - Class diagram

Figure 9 explains that all elements are based around the FileHandler class, which orchestrates the entire processing side of the workflow, taking the video files as an input (from the MainWindow/ElementHandler classes), extracting the video frames using the KeyFrameHandler, captioning the extracted key frames using the ImageCaptioningHandler, applying the search term to the caption using the LanguageHandler via the UI, then finally producing a report using the CompletionHandler.

5.4 Conclusion

Overall, the design phase of the project provided a comprehensive overview of the project's design process, paving the way for the implementation phase. The user interface was discussed first and then a combination of a simple GUI design, along with use case, package and class diagrams and their accompanying discussion created a solid foundation for a well-managed and efficient implementation process.

6 Implementation

6.1 Review of technology

Python 3.12 has been chosen to complete this project. Firstly, according to (StackOverflow, 2024), Python is the third most widely used programming language, which means there is plentiful resources online to assist any project. It is also beneficial for artificial intelligence (AI) projects due to widely available and well-documented libraries such as PyTorch and TensorFlow which allow implementation of AI projects with relative ease. PyTorch is known for its ease of use due to its user-friendliness and good debugging tools; this means it's generally popular among researchers and academics as it can be used for prototyping and experimentation (Atreya, 2024). Conversely, TensorFlow has a steeper learning curve and is more suited for industry and production systems (Atreya, 2024). For these reasons, PyTorch would be a better option due to its user-friendly nature. Due to both frameworks being widely utilised, it may be necessary to use either or both in some capacity, depending on which image description techniques are used; that is to say, neither option would be invalid. Python also includes an built-in library called unittest, which supports test automation – important for testing the final product (Python Software Foundation, 2025).

JetBrains PyCharm (Professional Edition) will be used as the primary IDE for developing the project due to its vast feature set, and good integration with testing tools and library management. PyCharm integrates Git, which is a version control system used for tracking changes in code, allowing for historical analysis of project progress. A GitHub repository will be setup (included on title page) to be used for this project. GitHub has a feature called 'Actions' which allow pipelines to be run, such as Pylint – a tool which checks code for errors and style issues making well-written, PEP8 compliant code much easier to achieve (Anand, 2023).

Due to the project requiring both processing and a GUI, several different libraries are needed to complete the project. Firstly, the PyQt6 library has been utilized to create a responsive and interactive GUI, its ease-of-use and documentation have enabled this to be a relatively painless experience. PyQt6 also includes great file processing functionality, which has been used to process and manage frames for detection. Qt Designer is included with PyQt6, which allows for comprehensive UI design, generating frameworks to be simply implemented into Python projects (The Qt Company, 2025).

Next, OpenCV and the Python Image Library (PIL) are needed for image processing. OpenCV will be used to convert the video into images, so that the image captioning model can process them, it can also be used for image analysis such as converting images to

greyscale and calculating their histograms (OpenCV team, 2024). PIL loads images into memory as objects, allowing easy access to properties such as the resolution and interfaces with captioning models (Lundh, 2011).

A widely used library for Natural Language Processing is Natural Language Toolkit (NLTK), which will aid the language side of the project. Specifically for processing WordNet's synsets, allowing for comparison of an inputted search term to the output of the image captioning algorithms. This will make the search more robust and user-friendly, as the project will be able to find relevant frames, even if the exact search term is not used (NLTK Project, 2024).

While looking for implementations of various models, Hugging Face became prevalent within search results. Hugging Face is a platform for open-source AI and ML model repositories providing access to well over a million pre-trained models, and more specifically, around 800 'image-to-text' models (Hugging Face, 2025), which is of particular relevance to this project.

For analysing both the automated and manual testing results, the R programming language in tandem with SQLite will be used. R is efficient for working with large, structured data such as tables, which is crucial for the automated testing that will be completed, which is expected to produce a vast amount of data in tabular form. The ggplot2 library provides excellent, highly customizable tools for creating data visualisations (Matloff, 2011). These features are particularly important as software such as Microsoft Excel does not allow tables (worksheets) larger than 1,048,576 rows in length (Microsoft, 2025). SQLite will also be used to store the data for analysis, as opposed to raw CSV files. SQLite is perfect for this, as it does not require a server to run, does not need configuration and provides SQL capabilities (Owens, 2006). R provides several libraries for SQL (Matloff, 2011) so that exactly the right data can be retrieved at the right time. The JetBrains DataSpell IDE will be used to perform the analysis as it integrates R and databasing tools for quick statistical development

6.2 Prototype/GUI

Firstly, the GUI needed to be created, while this has mainly been achieved within the prototyping stage, there is other considerations such as taking out any placeholder code and replacing that with code that's easily updatable to support the latter stages of the project.

The prototype was started by creating a simple UI based upon the design. The main consideration of this prototype was to make it as easy as possible to create the full product based upon the code within this prototype.

There are several elements within the user interface. Firstly, the ‘Select file...’ button will open the OS default file manager to allow the user to pick a video file for processing. There is an input field which will allow users to enter key words to look for in each frame, though for the prototype this is merely a placeholder. Once the user clicks ‘Find’, each frame of the video will be saved to a temporary directory. Each of these files is then displayed within the preview window with a delay of 1 second (to emulate object detection algorithm). At the bottom there is a progress bar and console, both of which allow progress tracking.

6.3 Data retrieval

Next, for automated testing, an appropriate dataset needed to be sourced. Several are available and these are discussed within the literature review section. For automated testing, the TextCaps dataset was sourced, as it contained a good middle ground of the number of images (which are presented as URLs and therefore don’t need downloading), along with around 5 reference captions for each image for comparison with the model’s generated output.

In contrast, for incremental along with manual testing, an appropriate set of images needed to be compiled. Due to the nature of this project, a variety of items should be considered, such as animals, objects, and video frames. Ultimately, the software will need to pick out objects from videos, which can have many variables such as occlusions, motion blur, and resolution (Sadkhan, et al., 2021). These images varied in complexity, allowing for a thorough analysis of the algorithms (see Appendix C.1). The final data retrieval step was to acquire some videos for testing the implementation. Following the same rules as the image collection, these videos also vary in complexity, such as a simple one camera video of an animal, camera footage above roads and dashcam footage (see Appendix C.2). All these videos were also duplicated and shortened into ~60 frame snippets, for quicker testing during implementation. Finally, a few testing videos of varying lengths were also created, by stitching together a variety of the sourced videos, for obvious scene changes within footage.

6.4 Image captioning analysis

Next, each object image captioning algorithm had to be analysed in several ways to make sure the most appropriate algorithm is selected for the project. The following metrics have been chosen to evaluate the captions of the chosen algorithms: BLEU, ROUGE and METEOR. These work by measuring the similarities between the reference captions and the generated ones. While these may be useful to measure the accuracy of the captions, other metrics will be necessary to measure the accuracy of object detection. Along with these automated testing methods, manual testing should take place, to check how useful the

information from the captioning algorithms is. Figure 10 shows a brief example of what the expected input/output using an image with a streetlamp and a plane in the sky might be, which emphasises what needs to be analysed.



Figure 10: Visual captioning overview

6.4.1 Automated Script Results

The main finding from this analysis and writing of an automated testing script is that there is no real benefit to training the algorithms again, all algorithms tested are pre-trained, meaning there is no need for extra time to be taken training algorithms for the purposes of this project as it is meant to be for general use.

The TextCaps dataset has been used within the automated testing script, along with the BLEU, METEOR and ROUGE-1, ROUGE-2 and ROUGE-L metrics. After this script was completed, data analysis was performed on the CSV output, and an average was calculated to get the overall results. Some image URLs are no longer available, so these have been skipped, along with some anomalies in images breaking the captioning algorithm. Table 2 shows general statistics, with Table 3 displaying the results from this testing averaged for easy understanding of the data.

Table 2: Testing statistics

Total results	Total time taken (HH:MM:SS)	Total images attempted	Total images analysed	Total missing images	Percentage images failed	Total algorithm errors
1,235,260	56:04:20	109,765	82,492	27,273	24.85%	424

Table 3: Results from automated model testing

Model	Metric	Result
OpenCLIP	BLEU	0.2301
OpenCLIP	METEOR	0.3477
OpenCLIP	ROUGE-1	0.3254
OpenCLIP	ROUGE-2	0.1067
OpenCLIP	ROUGE-L	0.2832
BLIP	BLEU	0.2406

BLIP	METEOR	0.3016
BLIP	ROUGE-1	0.2999
BLIP	ROUGE-2	0.094
BLIP	ROUGE-L	0.2676
Florence2	BLEU	0.0936
Florence2	METEOR	0.3632
Florence2	ROUGE-1	0.2551
Florence2	ROUGE-2	0.0832
Florence2	ROUGE-L	0.207

Figure 11 reveals that the OpenCLIP model generally outperforms BLIP and Florence2 in most metrics, with BLIP generally performing second highest and Florence2 trailing in all metrics with one outlier being METEOR. Florence2 outperforms in this metric because it focuses on semantic matching rather than exact word matches (Banerjee & Lavie, 2005), suggesting that it performs well in capturing the meaning of the reference captions, displaying a higher level of comprehension of the image, likely due to its ‘More detailed captioning’ (Xiao, et al., 2024) feature.

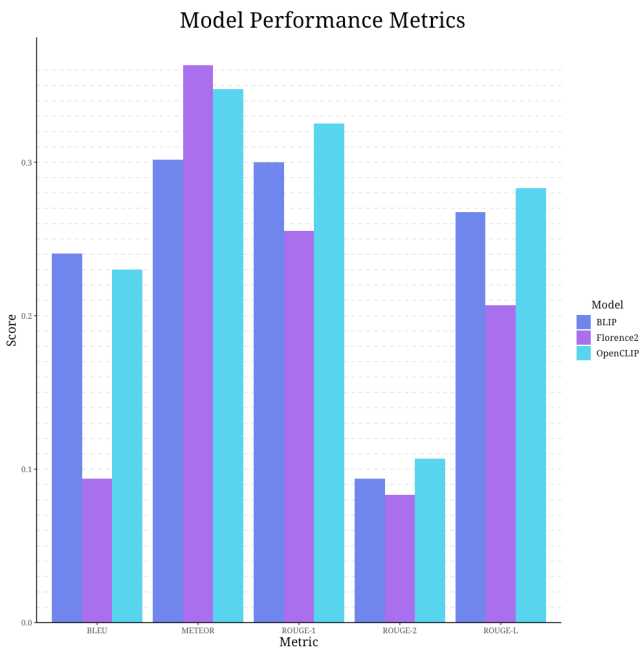


Figure 11: Grouped bar chart to visualise results from automated testing

Whilst these numbers represent a useful breakdown of the pros and cons of the models, due to the way these metrics are calculated, Florence2 may get overlooked as a valuable contender. Florence2’s METEOR score supports this hypothesis, thus manual testing needed to take place.

6.4.2 Manual Results

In addition to the automated testing above, another script was created to complete manual testing, using the manually compiled dataset of images. The aim of this was to determine

the quality of the output, specifically into the use case of this project. The results revealed some valuable insight into which algorithm to choose.

A simple script was created to run through each of the chosen models three times so that any issues would be apparent. Table 4 gives an averaged summary of the results, which reveals that the shorter the processing length per image, the shorter the caption is.

Table 4: Summary of manual model testing

Model	Avg. processing time per image (s)	Avg. caption length (chars)
OpenCLIP	1.8	47.25
BLIP	0.55	33.8
Florence2	5.94	477.3

These results were then visualised in Figure 12, further proving the hypothesis that the shorter the processing time, the shorter the caption. It also suggests that the Florence2 output captions contain much more detail than both OpenCLIP and BLIP.

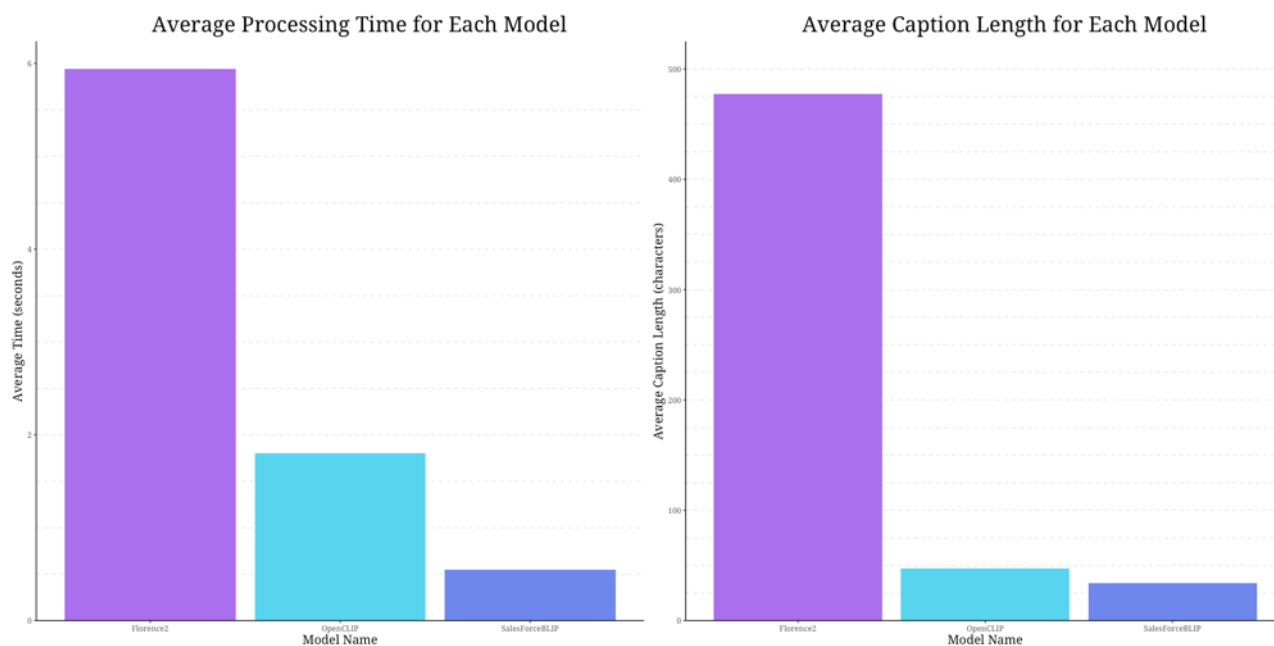


Figure 12: Avg. Processing time and caption length by model

In Table 5, an example from the results (available in Appendix B) have been taken to visualize how the models work. This clearly shows the difference between these models, with Florence2 going well above both OpenCLIP and BLIP in terms of the details picked from the images. Of particular interest in this case is that BLIP and OpenCLIP are incompatible with this project. For example, if a user wanted to search for the term 'car', this would not be possible with either of those models simply because the description is too short.

Table 5: Caption examples in manual model testing

Image	Model	Caption
	OpenCLIP	a bunch of trucks on a highway with a lot of traffic .
	BLIP	a long highway
	Florence2	The image shows a highway with multiple lanes of traffic. There are several trucks and cars on the road, including a red truck, a white truck, and a green truck. The highway is divided into two sections, with a toll booth on the left side and a sign that reads "Esso". On the right side of the highway, there are trees and hills in the background. The sky is overcast and the overall mood of the image is gloomy.

6.4.3 Concluding statement

Within this image captioning analysis, some key information was determined. Namely, that Florence2 will be the chosen algorithm to implement into the project. This is because of its ability to pick out much more detail in each image.

The next step will be to implement Florence2 into the project, by this point the GUI will have been finalized and placeholder code will be replaced with the image captioning algorithm processes. This will allow testing of the algorithms within the project itself, rather than externally using the compiled testing images. As the prototype will include video processing into frames, we can select one of these (likely the first, non-blank frame, for simplicity) to use as a proof of concept.

6.5 Frame selection

After implementing object detection on single images within the project, a method of selecting the most relevant images for detection needed to be found, there were two possible approaches to this, including using an existing keyframing technique, or a proposed method of a binary search-based approach to reducing the number of images which the image captioning algorithm will need to be applied to.

6.5.1 Binary Search Theory

As discussed in section 2.3.2, a way of overcoming the limitations of computing power could be based upon the binary search theory. However, upon further investigation, some issues with this approach became apparent.

Firstly, it would seem this approach would save resources, but in retrospect this is not the case. Implementing rewind functionality would add both complexity and may offset any efficiency gains due to having to re-import frames every time a rewind needed to take place. Another efficiency problem to note is that for every frame, the captioning algorithm

must be run, which is computationally expensive, as opposed to getting a subset of frames to test first, then running the algorithm second.

Additionally, the assumption that an object will always appear in its entirety may not hold true due to irregular movements within general video. For example, a captioning algorithm may pick up that there is a car in a video, even if said car is only partially on screen throughout the entirety of a video. Figure 13 explains this example. Each selected frame shows a partially visible red car which at no point is fully visible. The example captions may clearly show that this has been picked up, but if the software opted to rewind from the middle frame, it would never be able to find the car in its entirety, as this does not exist in the footage. This would lead to it getting stuck with no other frames to analyse, ultimately leading to a failure.

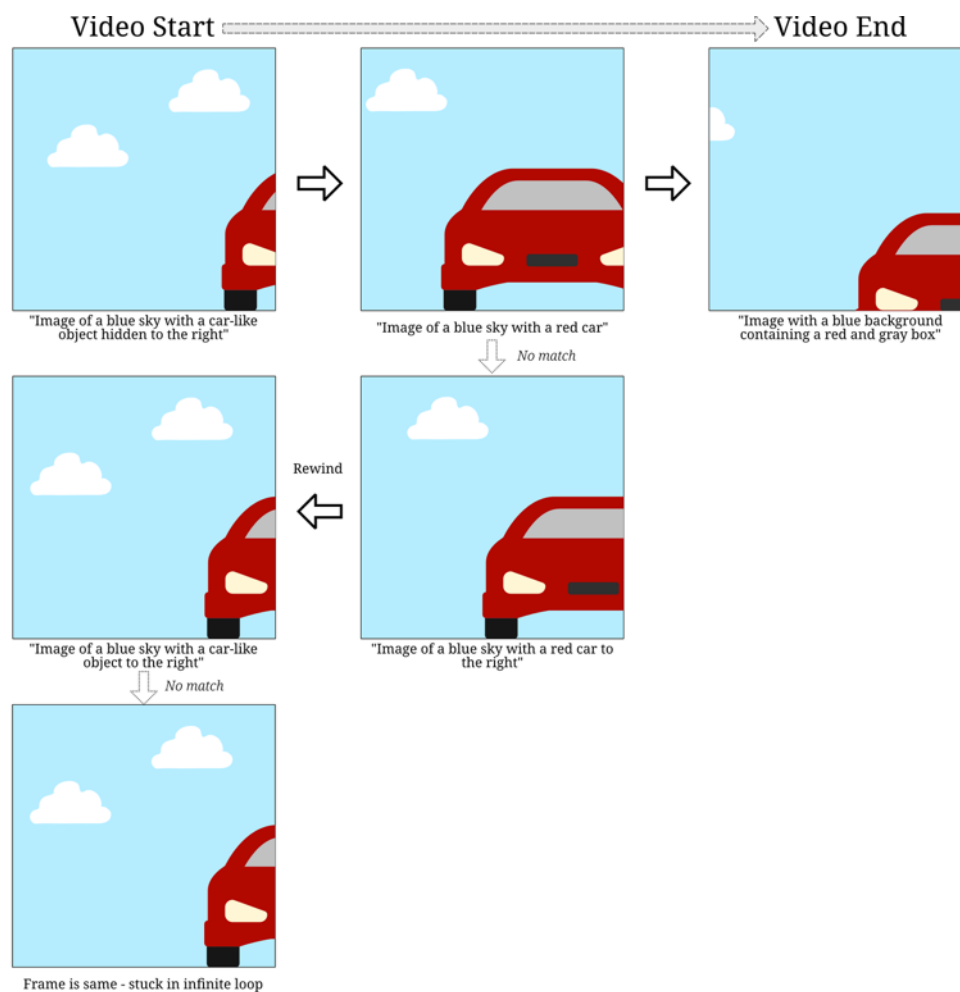


Figure 13: Binary Search Theory - Problem example

6.5.2 Key framing algorithm

A few methods of key framing were attempted. Firstly, using the FFmpeg library, which was an invalid method due to it selecting keyframes based on intra-frames, which are inserted periodically throughout a video (usually at fixed intervals) (FFmpeg, 2025), so do not have any focus on what each frame contains. Additionally, the Katna library was tested and was

found to be inadequate as it focuses on representative frames in a video, summarizing a video, rather than the details of each frame (KeplerLab, 2019).

After seeing that neither of these options were fit for purpose, key framing (discussed in section 2.3.1) was implemented due to its relative simplicity and its inclusion of a detailed explanation of the algorithm (Ghatak, 2016). Figure 14 shows this in action, every frame is converted to greyscale, then a histogram (with pixel values of black-white, 0-255 and their frequency) is generated. This is an extreme example of how a scene may change, but due to the vast difference in these images, the second frame is bound to be selected as a key frame in this instance.

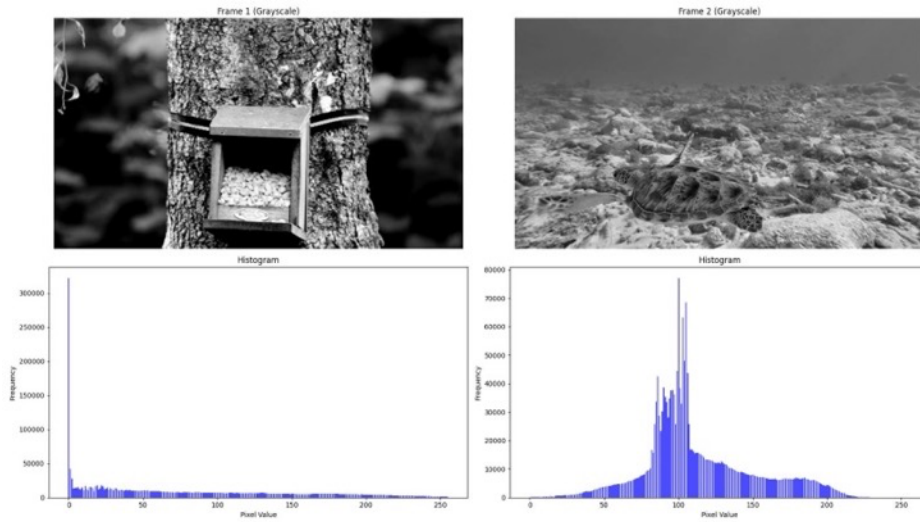


Figure 14: Key frame histogram with greyscale frames (top), and histograms (bottom)

6.6 Language handling

Using image captions in this way means that there are inevitably many extra, unnecessary words which need to be removed to find the specific word/object necessary to select an image as a match. Of course, an easy way of doing this would be to check every single word against the search term, but as discussed in section 2.6, this would not be reliable.

The first step is to get a set of all the synonyms/concepts of the search term, which involved WordNet synsets. NLTK includes WordNet and a simple loop retrieves a list of all relevant synset items, which is then converted into a set for use later, denoted as *search term synonyms*.

Next, the caption is processed using an exhaustive list of words to remove (pronouns, conjunctions, prepositions, and stop words) as well as the caption itself (excluding punctuation). Both elements are then converted to sets, which means that any duplicate words are automatically removed and the Boolean set theory operation *difference* ($-$, which is to say that all elements of set A are **not** in set B (Makinson, 2012)) can be used:

$$\text{relevant words from caption} = \text{original caption words} - \text{words to remove}$$

The last step brings the first two steps together by retrieving the intersection ($\cap$, which is to say that any item in set *A* **must be in** set *B* (Makinson, 2012)), of the remaining sets. This process produces a set of all words (and therefore objects) that are present in the frame:

$$\text{objects in image} = \text{search term synonyms} \cap \text{relevant words from caption}$$

6.7 Frame number retrieval

One of the final project requirements to be implemented was to return a timestamp of the frame in which the search term was found. Within Module 1 of the key framing algorithm, all the original frames from the video are extracted (Ghatak, 2016), which led to a graceful extension of this algorithm to include both the frame number (as needed by the algorithm) and the timestamp of the frame upon saving each frame. This meant that when a matching frame was found, the timestamp could simply be retrieved from the filename and displayed when needed.

6.8 Summary

The implementation chapter details the full development of this project. It utilises Python 3.12, along with an array of external libraries. The initial prototype featured a simple GUI with elements such as a file selector, search bar, preview window, progress bar and console. Next, automated and manual testing of each captioning algorithm took place and the results of this revealed that Florence2 would be the backbone of the project. The frame selection process was then covered leading to the implementation of a key framing algorithm (Ghatak, 2016). Next, utilizing NLTK and WordNet, the language handling section of the project was completed, with the final stage retrieving the frame number and timestamp when an object is found.

The final implementation combines the above elements to create functional software for object detection in videos using image captioning techniques.

7 Results

7.1 Key results

As can be seen from Table 6, most of the requirements from the original design have been met. All critical functional features, such as splitting videos into frames, specifying video files and language processing have been implemented, ensuring the project's core functionality, along with non-functional essentials such as the easy-to-use GUI, object-oriented design and file format limits.

Features improving user experience and system robustness, such as exception handling, process logging and adaptability to different file formats, are mostly implemented. However, some quality-of-life features such as processing time being longer than 30 minutes in some cases and video length warnings remain unimplemented.

Some lower priority features including a pause feature are partially implemented with multiple-word phrase searching and video length restrictions remain unimplemented with all rejected features being omitted.

Table 6: Feature listing ('F/NF' = 'Functional/Non-functional', 'I' = 'Implemented')

ID	Feature	F/NF	MoSCoW	I
1	Split a video into individual frames	F	Must	✓
2	An image captioning algorithm as the base of the project	F	Must	✓
3	One of the following two methods of reducing the number of images the image captioning algorithm needs to process:	F	Must	✓
3.1	Analysis of frames to retrieve the most relevant frames	F	(Must)	✓
3.2	An implementation of the binary search theory	F	(Must)	×
4	The ability to specify a video file	F	Must	✓
5	The ability to specify a word to search within the specified video file	F	Must	✓
6	Find synonyms for the user inputted word to search within captions	F	Must	✓
7	Provide the first timestamp for a word within a video	F	Must	✓
8	A processing length of no longer than 30 minutes	F	Should	×
9	A warning if the length of the video is too long (non-blocking)	F	Should	×
10	The ability to handle videos of different formats and resolutions	F	Should	✓
11	A highly accuracy image captioning method	F	Should	✓
12	Exception handling	F	Should	✓
13	Process logging	F	Should	✓
14	The ability to pause execution	F	Could	✓×
15	A final report in PDF format, with all relevant findings	F	Could	✓
16	A method of finding specific phrases within a video, not just a word (and its synonyms)	F	Could	×
17	An enforced limit on the length of the video importable to 2 minutes (or ~3600 frames)	F	Could	×

18	Multiple captioning methods/iterations of the same method	F	Won't	×
19	User-updatable parameters (such as number of iterations to perform)	F	Won't	×
20	A database system of results using MySQL	F	Won't	×
21	A simple Graphical User Interface (GUI)	NF	Must	✓
22	Implementation in an Object-Oriented software paradigm	NF	Must	✓
23	File format limits so that only compatible video files can be used	NF	Must	✓
24	High reliability	NF	Should	✓
25	High accuracy	NF	Should	✓
26	Feedback on progress:	NF	Should	✓
26.1	Progress bar	NF	Should	✓
26.2	Console/logging	NF	Should	✓
27	Feedback on progress	NF	Could	×
27.1	Frame counter	NF	Could	×
27.2	Total time elapsed	NF	Could	×
28	Parallelisation – Process multiple images at the same time	NF	Could	×
29	A video queuing system	NF	Won't	×

7.2 Unit testing

Unit testing is a process of testing individual components of a software application. These are tested in isolation, revealing any problems which may not be seen when simply testing the program in its final form. It focuses on the initial building blocks of the program, rather than testing the entire project (Myers, et al., 2012).

A test was written for each class within the processing package, which meant that an exhaustive test could be completed on the inner workings of the product. The user interface classes were not examined as they primarily serve as connectors to XML (markup) elements, while the processing package handles most of the computational workload.

While the full test cases and results are available in Appendix B.2, Table 7 shows the results for each suite from unit testing. Both positive and negative test cases have been included, to ensure code is reliable and contains a good level of exception handling.

Table 7: Averaged unit testing results

Suite	Tests completed	P/F	Time
TestCompletionHandler	9	P	27 ms
TestFileHandler	9	P	20.12 s
TestFrameDisplayer	6	P	21 ms
TestGlobalTools	5	P	4 ms
TestImageCaptioningHandler	7	P	108 s
TestKeyFrameHandler	9	P	337 ms
TestLanguageHandler	19	P	1.4 s
Totals	64	7P 0F	129.96 (s)

7.3 Manual testing

There were two elements to the manual testing process. Firstly, a list of items to test was created in a spreadsheet, which were repeated three times (final run shown in Table 8, full results in Appendix B.3.1). This gave a general overview of the product in its final form, from an end-user perspective.

Table 8: Manual testing overview

Data type	Test objective	Test Result (P/F)
Normal	File import	P
Erroneous	File import	P
Normal	Short video (2 frames) import	P
Normal	Medium video (2 seconds/60 frames) import	P
Normal	Long video (1+ minutes) import	P
Normal	Single word search term (turtle_2s.mp4)	P
Erroneous	Single word search term (turtle_2s.mp4)	P
Normal	Single word search term (truck) (unfocused_motorway_2s.mp4)	P
Erroneous	Multiple word search term (unfocused_motorway_2s.mp4)	P
Normal	Single word synonym (motortruck) (unfocused_motorway_2s.mp4)	P
Erroneous	Multiple word synonym (unfocused_motorway_2s.mp4)	P
Normal	PDF output on successful find	P
Erroneous	PDF output on unsuccessful find	P
Normal	Deletion of temporary files on open	P
Normal	Data logging	P

Next, a performance test was completed using a set of four videos. These varied in length, from the shortest consisting of only four frames, to a two-minute extended video featuring a mix of scenes to test the limits of the program. Table 9 shows that for a 20 second video, it takes around 23 minutes to find a specified object within a video. The second result was only completed once, due to the unreasonable time it took to process, which is beyond the scope of this project, with the final test (result starred) being cancelled after over 3 hours had passed, and only around 75% of the frames being saved.

Table 9: Averaged completion time by video

Video	Runs	Avg. completion time	Avg. percentage of frames processed
four_frames.mp4	3	17.3 secs	50%
turtle_30s.mp4	1	1hr 23 mins 1 sec	20.9%
mixed_scenes_20s.mp4	3	23 mins 56 secs	6.6%
mixed_scenes_2m.mp4	1	3 hrs 7 min 33 secs*	N/A

7.4 Feature Demonstration

Upon compilation of the software, the user is greeted with the application GUI. The GUI contains several elements. At the top, the 'Select File' button along with the accompanying field will automatically fill with the file name when a user selects a file. Next to this, there is a field for a user to input their object to find within the selected video. On the right, there is a 'Find' button which starts the process. If a user does not select a file, or input a search term, upon clicking the 'Find' button, an error message will appear in the console located at the bottom of the GUI. Next, there is a large preview window which shows any processing in real-time such as the key framing routine, and captioning process. Below this, there is a 'Continue' button, which simply continues execution (the process stops for 10 seconds when a match is found) if clicked.

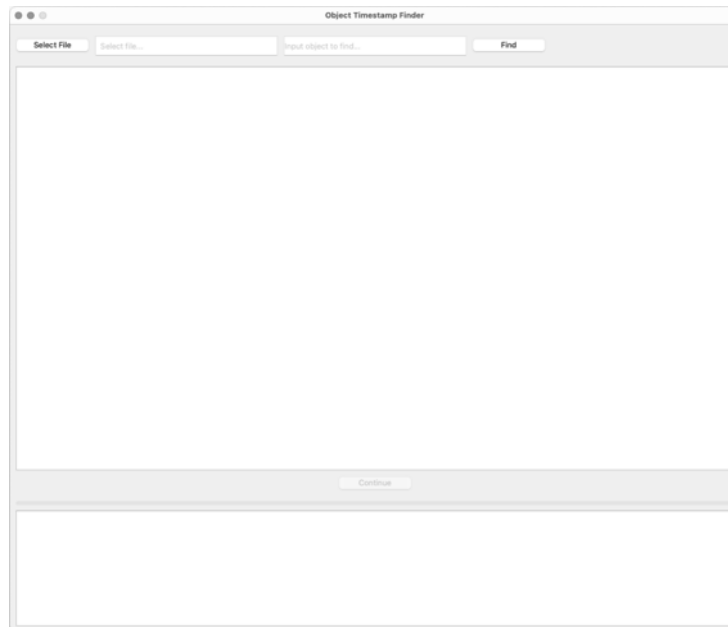


Figure 15: Demo - Initial User Interface

Next, the user is expected to select a file, where a file is inputted using the host operating systems default file manager. The file must be either a *.mp4 file, or an *.mkv file, thus only these files will be selectable within the picker.

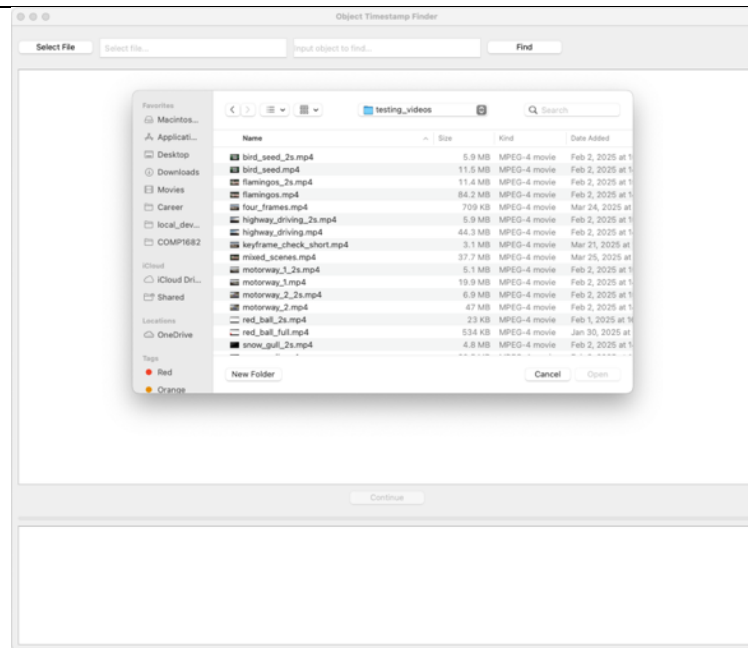


Figure 16: Demo - Select File

After the video file is selected, the user must enter a single-word search term within the search field, and then click 'Find'. This starts the processing, where the frames are split from the video, as indicated by the console.

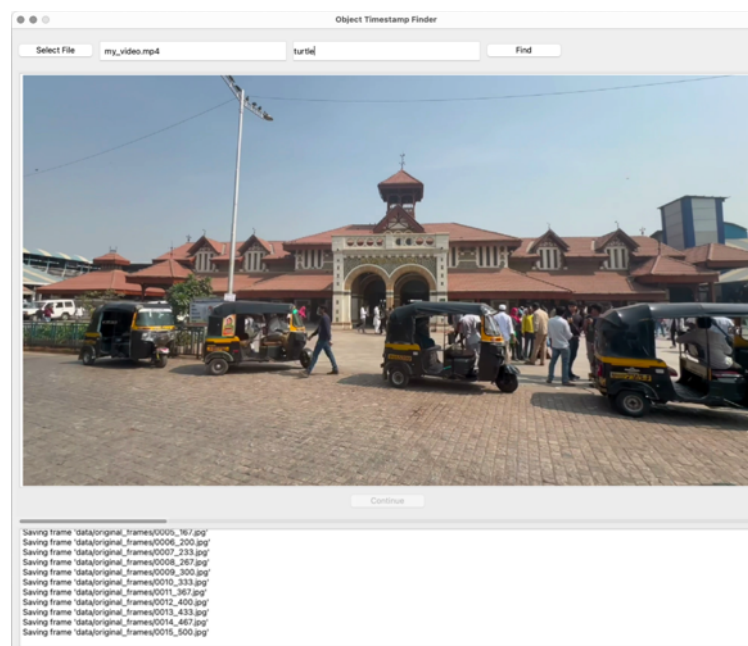


Figure 17: Demo - Search term entry

At this point, all the frames have been saved, and the key framing process starts. In the preview window, this process is displayed. The greyscale images indicate the frames which are currently being compared, and underneath the histogram of that image is displayed.

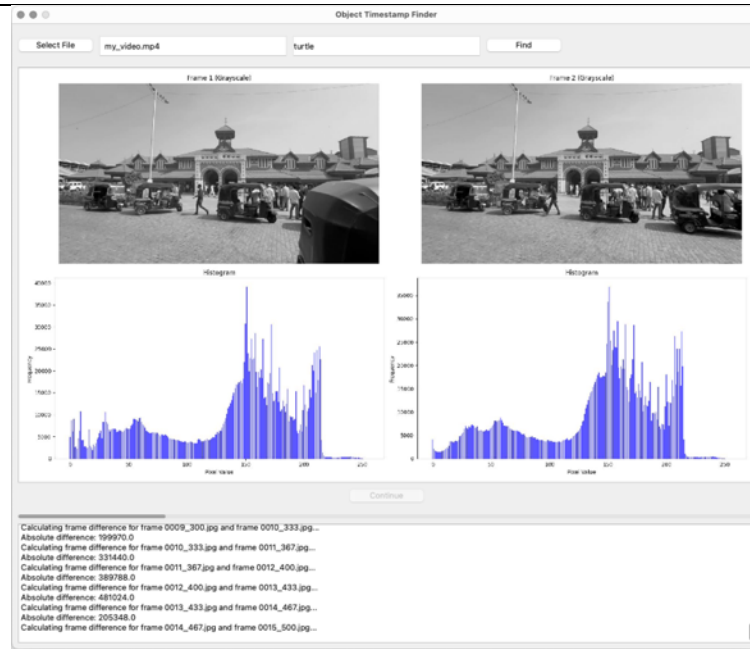


Figure 18: Demo - Key framing process

After the keyframes are selected, the program moves onto captioning. Each keyframe is run through the image captioning algorithm and the output of this is indicated in the console. When a matching word is found within the caption, the program pauses for 10 seconds, to display the timestamp at which the word is found. The user can click 'Continue' to skip the wait.

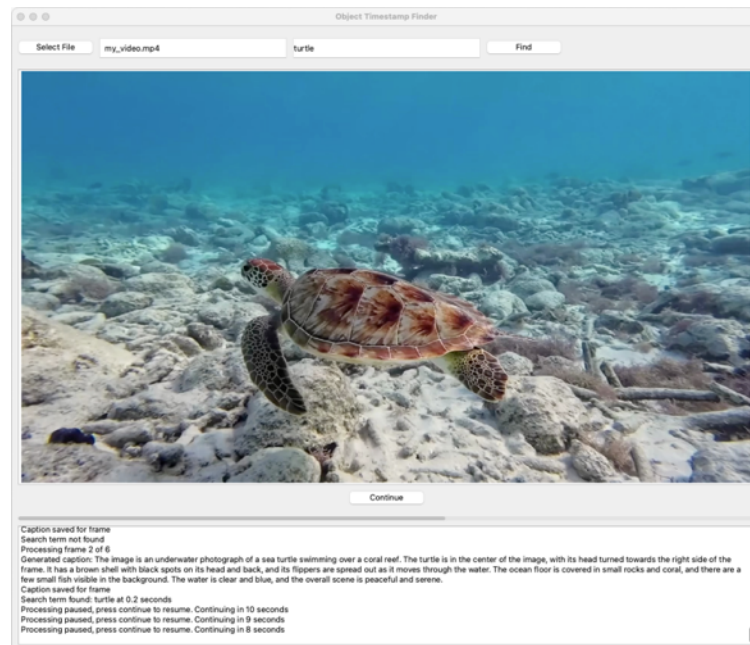


Figure 19: Demo - Found matching frame

After all frames have been captioned, a modal window appears to ask the user whether they want a PDF report to be generated. It features a 'Choose' button which brings up the host operating systems default file manager, along with 'Yes' and 'No' options, to either

create a report, or simply close the window so the user can look through the console history to find any relevant details.

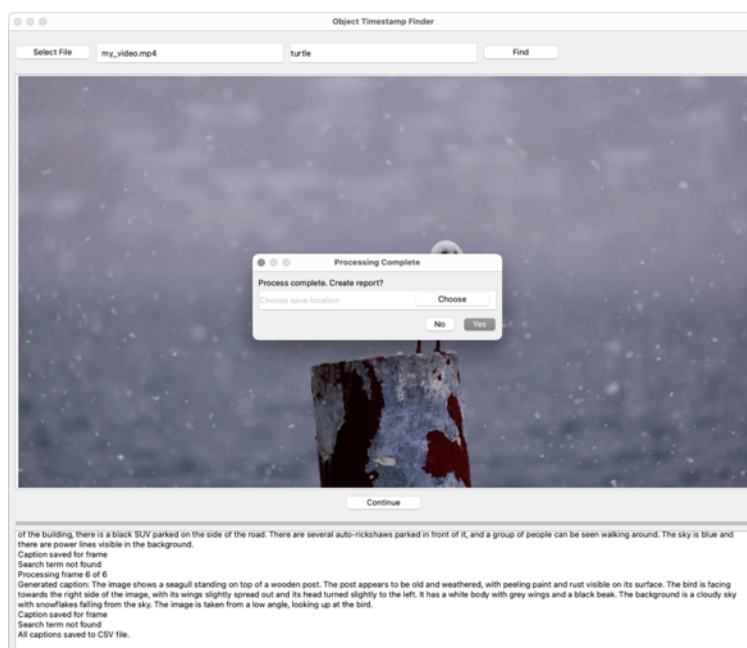


Figure 20: Demo - Report modal

If a user decides they would like a PDF report, they first click the ‘Choose’ button to open the file manager and pick a directory in which they want to save the report.

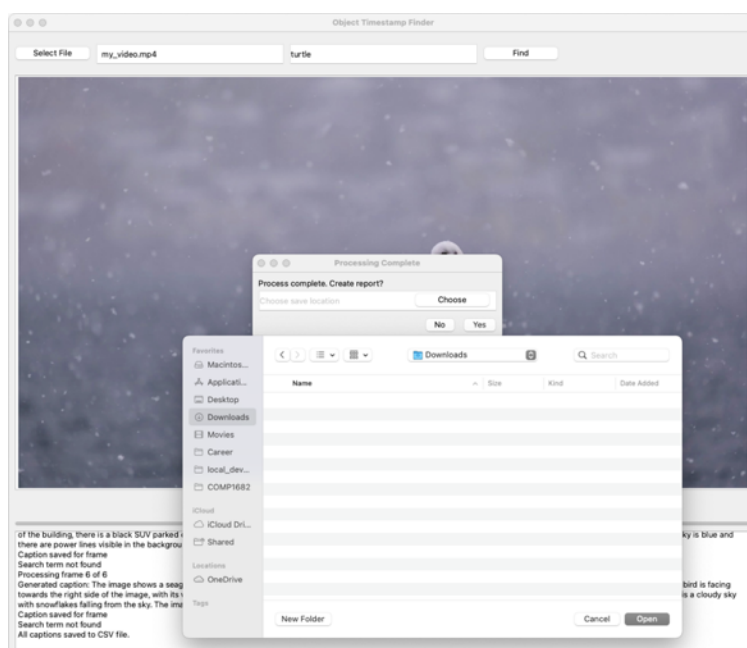


Figure 21: Demo - Select report directory

Once the directory is picked, the user selects ‘Yes’, and the report is generated in the chosen directory, completing the entire process.

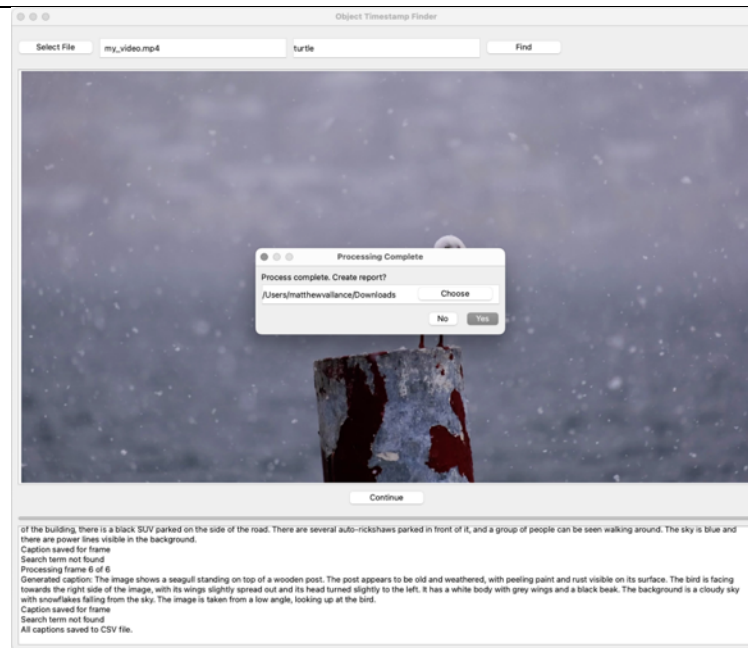


Figure 22: Demo - Confirm report directory

7.4.1 Output

Assuming the user selects ‘Yes’ in the report modal, a standard A4 size PDF is generated in their chosen output location. The report includes the title of the original file (excluding the file extension) and the inputted search term. For each matching frame, a new page is added with the frame filename, a copy of the frame and a table with the caption, timestamp and frame number listed.

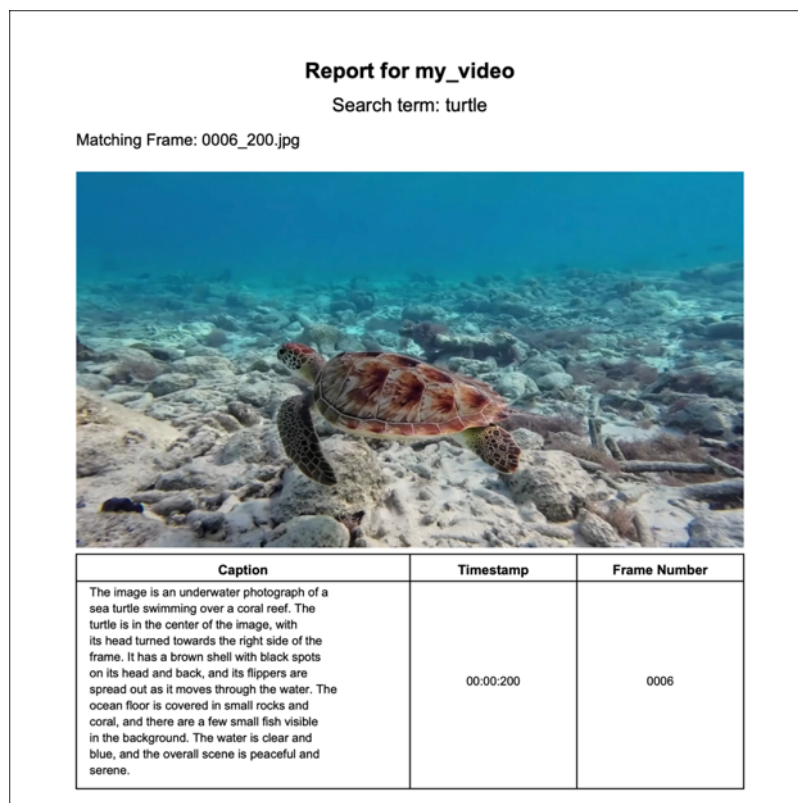


Figure 23: Demo - Completion report PDF output

7.5 Summary

In summary, the results of this project demonstrate that the majority of the original design requirements have been successfully implemented, including both critical processing functionality, along with non-functional essentials.

Unit testing revealed that individual components are reliable and contain a good level of exception handling. Manual testing showed that it performs well with various video lengths and search terms, but longer videos present a significant issue in processing time.

Finally, the operation of the software was discussed, moving through each of the features in order, demonstrating the complete workflow of the final product. These three elements reinforce significant outcomes of the project, highlighting the successful implementation of core functionalities, and the areas which could be improved in future iterations.

8 Evaluation

8.1 Outcomes

From the results chapter, the project objectives have mostly been met (see Introduction 1.2). Firstly, various image captioning techniques were investigated and the objectively best one, Florence2 was selected and successfully implemented, enabling individual frames to be captioned with a high level of detail.

Next, a key framing method was fully implemented, significantly reducing the amount of data taken from the original video, therefore significantly reducing the processing required (in most cases, see Limitations below), thus allowing consumer-grade hardware to be used. This proved to be very effective and from testing could remove over 90% of redundant data (see Results 7.3).

In addition, language processing was implemented. Upon starting the processing, a routine is applied to the search term to find all related words using WordNet, therefore finding objects no matter how they are described. A prime example of this is ‘truck’ and ‘motortruck’ (see Results 7.3); both were applied to the same video, and the same frame at the same timestamp was returned as a match.

Finally, the exact timestamp in which the search term is found is accurately returned to the millisecond. Though this differs from the original goal of the timestamp being fully visible on-screen (see Appendix A.2/A.7), upon further investigation this was not plausible (see Implementation 6.5.1). In addition, these results are returned to the user in a meaningful way via a PDF report.

8.2 Limitations

As can be seen from the results section, there are a few notable limitations. Firstly, only single word search terms can be entered, this was partially due to limitations in the timescale, and partially due to limitations in initial planning. In retrospect, initial testing should have been completed (much like model testing) to test the implementation of multi-word synonyms using WordNet or other tools, leading to a more efficient (therefor more complete) implementation.

As part of the computational constraints’ objective, the original goal outlined in the proposal (Appendix A.1/A4.4/A.7) was to be able to input a video of a maximum length of 2 minutes to be completed in 30 minutes. As can be seen from the manual testing results (see

Results 7.3), a 20 second video (containing 606 frames) takes approximately 30 minutes to complete.

Another limitation is that single scene videos, such as the ‘turtle_30s’ example in Table 9, return a much higher percentage of key frames, this is due to much of the frames being similar, thus many of the frames contain the same object (see Appendix B.4). In retrospect, a way of stopping execution completely when an object was found would have been beneficial.

8.3 Unexpected challenges

8.3.1 Poor performance

A particularly interesting and surprising challenge was disk speed. After discovering that the disk usage was extremely high (causing the occasional crash) when running the software, a short sleep of 0.1s was implemented on each frame saved. While this helped solve the problem, RAM seemed to be the bottleneck during the full test. Table 10 shows usage at select points, with screenshots available in Appendix B.5, both of which illustrate that it uses a vast amount of memory, and better management of resources could have been implemented.

Table 10: RAM usage

Point of reference	Frame retrieval	Start of key framing calculations	End of key framing calculations	Start of report generation
RAM usage	13.03 GB	595.4 MB	7.52 GB	19.29 GB

This meant that a 2-minute video could not be processed in a reasonable timescale. Once the RAM usage is out of control (at around 800 frames) each frame takes around 30 seconds to be saved to the drive, in addition to the remaining processes to be completed.

8.3.2 Automated testing

An unexpected challenge and therefore limitation of this section was that to gain a full picture of model performance, the metrics used should have included CIDEr and SPICE (Al-Shamayleh, et al., 2024). These were not included due to them both being around 10 years old and requiring Java 1.8+ (originally released early 2014), which lead to available codebases being incompatible with even moderately recent versions of Python. Upon further investigation, the code would have required significant rewriting, which would have delayed project completion.

A dataset needed to be sourced as part of automated testing; several of which were discussed previously (see Literature Review 2.4). Once retrieved, the issue of missing

images became apparent (see Implementation 6.4.1). Due to the TextCaps dataset being around 5 years old (Sidorov, et al., 2020), some image URLs were missing; therefore, adjustments needed to be made to account for any missing data from the original dataset. Initially these were skipped, however it became clear that it would be beneficial to know how much data was skipped, so that any presented findings were accurate to the data which was processed.

8.3.3 Unit testing

While the results presented the final run-through of each unit test. Initially, all positive test cases were completed faultlessly, but the negative test cases revealed some key issues. Firstly, the software did not check to see if any frames were found before generating the report, which meant that the user would be asked if they wanted to generate a report, even if no valid frames were found. In addition, it revealed that within the caption splitting routine in the language processor, blank strings and punctuation needed removing, while this was not inherently an issue, it will have saved some processing resources in the long run. Error handling was implemented as a result, and upon re-running these tests, everything passed.

8.4 Summary

From the detailed analysis in this chapter, it can be deduced that the product generally achieves the original goal of retrieving objects from short videos, though there are notable shortcomings which could be resolved in future iterations.

All four objectives set out in the introduction have been achieved, bringing all the elements together to create a working product. However, there are significant limitations in terms of the performance. A 20 second video takes around 30 minutes to complete, which is far from the original goal of a 2-minute video processed in 30 minutes.

There were also some unexpected challenges during development such as poor performance, particularly in terms of RAM usage. Furthermore, some automated testing metrics were excluded when analysing captioning models due to their antiquated code. Additionally, large amounts of data were missing from the testing dataset, requiring adjustments to be made, and limiting the available data for analysis. Finally, unit testing posed some challenges resulting in implementation of error handling.

9 Legal, Social and Professional Issues

Firstly, some legal issues had to be considered. Due to this project requiring vast amounts of external libraries and models, the issue of code licencing became apparent. Upon researching this, all portions of the project which were required for the final product were open source, clearing any legal issues which may have arisen from the use of external codebases. An exception to this is the use of the key framing algorithm proposed by (Ghatak, 2016), due to code not being provided with this paper, the theories behind this needed to be implemented, thus an in-code comment has been included to properly acknowledge this paper in the relevant sections.

A thoroughly researched social and ethical issue among any artificial intelligence model is that of social biases. Artificial Intelligence models can unfairly treat characteristics like gender, ethnicity, or income, which is known as digital discrimination (Ferrer, et al., 2021). While this is true of any model, it can be applied directly to image captioning. Research has definitively found several issues, such as biases in representation with lighter-skinned males being overly prevalent and darker-skinned females being severely underrepresented. These racial biases seem to manifest in captions through disparities in descriptors, performance and word choice – particularly in regards to racial terms along with racial slurs (Zhao, et al., 2021). Due to this project aiming to simply find objects in a video, it can be assumed that digital discrimination does not meaningfully apply in this context, that is not to say that this software could not be abused by bad actors, but this falls beyond the scope of this implementation.

Finally, both the IEEE Code of Conduct (IEEE, 2014) and the BCS Code of Conduct (BCS, 2022) contain similar principals that members must abide by. The most important point which directly relates to this project is that of acting with integrity and honesty in professional activities. Both codes state that members are expected to avoid misrepresentation, false claims and unethical behaviour, and that they must hold accountability for their actions.

10 Conclusion and Future Work

10.1 Conclusion

The project set out to harness existing technologies to create a user-friendly software package for finding the exact timestamp at which an object is visible within a video. This implementation successfully brought together all the key elements for a complete product. The Florence2 model generates a detailed description of frames selected through a key framing technique, enabling users to input a video along with a search term and receive a reliable timestamp indicating where that term appears.

The extensive literature review provided an exhaustive foundation and set up for a comprehensive requirements analysis. Next, the design chapter detailed the prototype user interface along with the system architecture defined by UML diagrams. Based on this, the implementation chapter detailed the entire implementation process, starting with a review of the technology that would be used within the project and ending with a short explanation of the user interface. At this point, the extensive analysis of image captioning models became the focus, justifying the reasons why Florence2 was the best algorithm for this project. An explanation on why the binary search theory outlined in the proposal would not be the correct approach was next, leading to the implementation of a key framing algorithm instead. The penultimate stage was search and language handling using WordNet to remove unnecessary extra detail from captions with the final stage implementing accurate timestamp retrieval by saving that information onto the file name for each extracted frame. The evaluation chapter then gives an analytical description of results, which were presented in an easy to digest format. Finally, some legal, social and ethical issues were discussed with an exposition on software engineering principals upheld during this project before ending with the concluding chapter.

The evaluation chapter revealed several key results of the implementation. Notably, it demonstrated that integrating an image captioning model with a key framing algorithm successfully enables the precise retrieval of timestamps for objects in a short video. Furthermore, it revealed that the application effectively identified and reported relevant frames with a good level of accuracy and that synonyms of words can also be detected, demonstrating that a wide variety of terminology can be used to find the same object. Conversely, it also revealed some limitations, such as only being able to process videos less than 20 seconds long within 30 minutes. As opposed to the original goal of 2-minute videos in 30 minutes. This was caused by disk speed and RAM issues. In addition, only single word search terms can be entered to be found.

Overall, the project is considered successful. While the implementation does not enable multi-word search terms, all four objectives set out in the introduction have been largely achieved, bringing all the elements together to create a well-rounded, working product. The choice of implementing Florence2, a key framing algorithm and language tools aided by exhaustive testing throughout implementation and upon completion, has led to a successful project with scope for future enhancements.

10.2 Future work

Whilst the completed project achieves all the objectives and generally achieves the original aim of retrieving objects from short videos, some improvements could be made in subsequent iterations. The immediate next steps to fully achieve the intended goal as set out in the proposal (see Appendix A.1/A4.4/A.7), would be to allow multiple words in the search term, and to fix the RAM issues (see Evaluation 8.3.1) to achieve the goal of a 2-minute video being processed in 30 minutes.

Following completion of these issues, this solution could be applied in a different context. Specifically, the methodology applied (combining an image captioner with key framing in videos) for consumers could be applied in a commercial setting. For example, in an investigation of a convenience store robbery, at the exact time and what was taken could be determined via multiple camera feeds, such as the door and shelves where merchandise was taken.

10.3 Closing remarks

From the proposal, a general overview of the subject matter was gathered, leading to extensive research being carried out which has laid the groundwork for a comprehensive design and implementation. Overall, the knowledge gained paired with a generally well thought out implementation has led to a successful project, with a clear-cut path for future work.

11 References

- Agrawal, H. et al., 2019. *nocaps: novel object captioning at scale*. Seoul, South Korea, IEEE Computer Society.
- Al-Shamayleh, A. S. et al., 2024. A comprehensive literature review on image captioning methods and metrics based on deep learning technique. *Multimedia Tools and Applications*, Volume 83, pp. 34219-34268.
- Anand, R., 2023. *Using 'Pylint' for Python Code Analysis in GitHub Actions*. [Online] Available at: <https://rohan-anand.hashnode.dev/using-pylint-for-python-code-analysis-in-github-actions> [Accessed 8 April 2025].
- Atreya, M., 2024. *PyTorch vs. TensorFlow: A Comprehensive Comparison in 2024*. [Online] Available at: <https://docs.rafaq.co/blog/2024/09/16/pytorch-vs-tensorflow-a-comprehensive-comparison-in-2024/> [Accessed 3 November 2024].
- Banerjee, S. & Lavie, A., 2005. *METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments*. Ann Arbor, Michigan, USA, Association for Computational Linguistics.
- Barbu, A. et al., 2019. ObjectNet: A large-scale bias-controlled dataset for pushing the limits of object recognition models. In: H. M. Wallach, et al. eds. *Advances In Neural Information Processing Systems (NeurIPS 2019)*. Vancouver, BC, Canada: Neural Information Processing Systems Foundation, pp. 9448-9458.
- BCS, 2022. *BCS Code of Conduct*. [Online] Available at: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/> [Accessed 10 April 2025].
- Cherti, M. et al., 2023. *Reproducible scaling laws for contrastive language-image learning*. Vancouver, BC, Canada, IEEE, pp. 2818-2829.
- Cobb, C. G., 2015. *The Project Manager's Guide to Mastering Agile: Principles and Practices for an Adaptive Approach*. 2nd ed. Hoboken, NJ, USA: Wiley.
- Deng, J. et al., 2009. *ImageNet: A Large-Scale Hierarchical Image Database*, Miami, FL, USA: IEEE.
- Deshmane, P., 2017. *MRKFE: Designing cascaded map reduce algorithm for key frame extraction*. Chennai, India, IEEE.

- Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K., 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1, pp. 4171-4186.
- Everingham, M. et al., 2015. The PASCAL Visual Object Classes Challenge: A Retrospective. *International Journal of Computer Vision*, 111(1), pp. 98-136.
- Ferrer, X. et al., 2021. Bias and Discrimination in AI: A Cross-Disciplinary Perspective. *IEEE Technology and Society Magazine*, 40(2), pp. 72-80.
- FFmpeg, 2025. *Documentation*. [Online]
Available at: <https://ffmpeg.org/documentation.html>
[Accessed 18 April 2025].
- Ghatak, S., 2016. Key-Frame Extraction Using Threshold Technique. *International Journal of Engineering Applied Sciences and Technology*, 1(8), pp. 51-56.
- Google, 2025. *Detect Multiple Objects | Cloud Vision API | Google Cloud*. [Online]
Available at: <https://cloud.google.com/vision/docs/object-localizer>
[Accessed 15 January 2025].
- Google, 2025. *Object Detector Guide | Vertex AI Vision | Google Cloud*. [Online]
Available at: <https://cloud.google.com/vision-ai/docs/object-detector-model>
[Accessed 25 January 2025].
- Hugging Face, 2025. *Models - Hugging Face*. [Online]
Available at: https://huggingface.co/models?pipeline_tag=image-to-text
[Accessed 13 April 2025].
- IEEE, 2014. *IEEE Code of Conduct*. [Online]
Available at: <https://www.ieee.org/about/corporate/governance/code-of-conduct.html>
[Accessed 10 April 2025].
- Kaur, S., Kaur, L. & Lal, M., 2024. An effective Key Frame Extraction technique based on Feature Fusion and Fuzzy-C means clustering with Artificial Hummingbird. *Scientific Reports*, Volume 14.
- KeplerLab, 2019. *Katna documentation*. [Online]
Available at: <https://katna.readthedocs.io/en/latest/>
[Accessed 18 April 2025].
- Krishna, R. et al., 2016. *Visual Genome: Connecting Language and Vision Using Crowdsourced Dense Image Annotations*, s.l.: Kluwer Academic Publishers.
- Krizhevsky, A., 2009. *Learning Multiple Layers of Features from Tiny Images*, s.l.: s.n.

- Kurdi, M. Z., 2016. *Natural Language Processing and Computational Linguistics 1*. 1st ed. London, United Kingdom: Wiley-ISTE.
- Li, J., Li, D., Xiong, C. & Hoi, S., 2022. *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. Baltimore, Maryland, USA, PMLR.
- Lin, C.-Y., 2004. *ROUGE: A Package for Automatic Evaluation of Summaries*. Barcelona, Spain, Association for Computational Linguistics.
- Lin, T.-Y. et al., 2014. *Microsoft COCO: Common Objects in Context*, Zurich, Switzerland: Springer, Cham.
- Lundh, F., 2011. *Pillow (PIL Fork) 11.2.1 documentation*. [Online]
Available at: <https://pillow.readthedocs.io/en/stable/reference/Image.html>
[Accessed 16 April 2025].
- Makinson, D., 2012. *Sets, Logic and Maths for Computing*. 2nd ed. London, United Kingdom: Springer-Verlag London Limited.
- Matloff, N., 2011. *The Art Of R Programming: A Tour of Statistical Software Design*. 1st ed. San Francisco, CA, USA: No Starch Press, Inc..
- McCrae, J. P. et al., 2019. *English WordNet 2019 – An Open-Source WordNet for English*. Wroclaw, Poland, Global Wordnet Association, p. 245–252.
- Michaels, P., 2024. *I just tried Apple Intelligence photo search in iOS 18 — and I'm surprised with the results*. [Online]
Available at: <https://www.tomsguide.com/phones/iphones/i-tried-out-apple-intelligences-tool-for-photo-searches-and-was-surprised-by-what-i-found>
[Accessed 25 January 2025].
- Microsoft, 2025. *Excel specifications and limits*. [Online]
Available at: <https://support.microsoft.com/en-us/office/excel-specifications-and-limits-1672b34d-7043-467e-8e27-269d656771c3>
[Accessed 4 April 2025].
- Miller, G. A. et al., 1990. Introduction to WordNet: An On-line Lexical Database. *International Journal of Lexicography*, 3(4), pp. 235-244.
- Myers, G. J., Sandler, C. & Badgett, T., 2012. *The Art of Software Testing*. 3rd ed. Hoboken, New Jersey: John Wiley & Sons, Inc..
- NLTK Project, 2024. *NLTK :: Natural Language Toolkit*. [Online]
Available at: <https://www.nltk.org/>
[Accessed 15 April 2025].

- OpenCV team, 2024. *OpenCV: Introduction*. [Online]
Available at: <https://docs.opencv.org/4.10.0/d1/dfb/intro.html>
[Accessed 16 April 2025].
- Owens, M., 2006. *The definitive guide to SQLite*. 1st ed. Berkeley, CA, USA: Apress.
- Papineni, K., Roukos, S., Ward, T. & Zhu, W.-J., 2002. *BLEU: a Method for Automatic Evaluation of Machine Translation*. Philadelphia, PA, USA, Association for Computational Linguistics.
- Python Software Foundation, 2025. *3.12.10 Documentation*. [Online]
Available at: <https://docs.python.org/3.12/>
[Accessed 15 April 2025].
- Radford, A. et al., 2021. *Learning Transferable Visual Models From Natural Language Supervision*, s.l.: s.n.
- Redmon, J., Divvala, S., Girshick, R. & Farhadi, A., 2016. *You Only Look Once: Unified, Real-Time Object Detection*. Las Vegas, Nevada, USA, IEEE.
- Robertson, S. & Robertson, J., 2013. *Mastering the Requirements Process*. 3 ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional.
- Roboflow, 2025. *Roboflow: Computer vision tools for developers and enterprises*. [Online]
Available at: <https://roboflow.com/>
[Accessed 6 April 2025].
- Sadkhan, A. S. B., Talebiyan, S. R. & Farzaneh, N., 2021. *An Investigate on Moving Object Tracking and Detection in Images*, Babil, Iraq: IEEE.
- Sedgewick, R., 1983. *Algorithms*. 1st ed. Providence, Rhode Island, USA: Addison-Wesley Publishing Company.
- Seidl, M., Scholz, M., Huemer, C. & Kappel, G., 2015. *UML @ Classroom: An Introduction to Object-Oriented Modeling*. 1 ed. Linz, Austria: Springer Cham.
- Sharma, H. & Padha, D., 2023. A comprehensive survey on image captioning: from handcrafted to deep learning-based techniques, a taxonomy and open research issues. *Artificial Intelligence Review*, 56(11), pp. 13619-13661.
- Sharma, P., Ding, N., Goodman, S. & Soriccut, R., 2018. *Conceptual Captions: A Cleaned, Hypernymed, Image Alt-text Dataset For Automatic Image Captioning*. Melbourne, Australia, Association for Computational Linguistics.
- Sidorov, O., Hu, R., Rohrbach, M. & Singh, A., 2020. *TextCaps: a Dataset for Image Captioning with Reading Comprehension*. Glasgow, UK, Springer, Cham.

- Sinulingga, H. R. & Kong, S. G., 2023. Key-Frame Extraction for Reducing Human Effort in Object Detection Training for Video Surveillance. *Electronics*, 12(13), p. 2956.
- StackOverflow, 2024. *2024 Developer Survey*. [Online]
Available at: <https://survey.stackoverflow.co/2024/technology/>
[Accessed 17 January 2025].
- The Qt Company, 2025. *Tools - Qt for Python*. [Online]
Available at: <https://doc.qt.io/qtforpython-6/tools/index.html#package-tools>
[Accessed 15 April 2025].
- Wang, A. et al., 2024. *YOLOv10: Real-Time End-to-End Object Detection*. Vancouver, Canada, Curran Associates, Inc..
- Wightman, R., Beaumont, R., Gordon, C. & Shankar, V., 2025. *mlfoundations/open_clip: An open source implementation of CLIP*. [Online]
Available at: https://github.com/mlfoundations/open_clip
[Accessed 1 May 2025].
- Xian, Y., Lampert, C. H., Schiele, B. & Akata, Z., 2019. Zero-Shot Learning—A Comprehensive Evaluation of the Good, the Bad and the Ugly. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9), pp. 2251-2265.
- Xiao, B. et al., 2024. *Florence-2: Advancing a Unified Representation for a Variety of Vision Tasks*. Seattle, WA, USA, IEEE.
- Zhao, D., Wang, A. & Russakovsky, O., 2021. *Understanding and Evaluating Racial Biases in Image Captioning*. Montreal, QC, Canada, IEEE.
- Zhong, Y. et al., 2022. *RegionCLIP: Region-based Language-Image Pretraining*. New Orleans, LA, USA, IEEE.
- Zhou, C., Guo, Y., Lv, Q. & Yuan, J., 2024. *Optimizing Object Detection via Metric-driven Training Data Selection*. Seattle, USA, IEEE.
- Zhou, L. et al., 2020. *Unified Vision-Language Pre-Training for Image Captioning and VQA*. New York, AAAI Press.
- Zhuang, F. et al., 2021. A Comprehensive Survey on Transfer Learning. *Proceedings of the IEEE*, 109(1), pp. 43-76.

Appendix A – Project proposal

Note: References within this section are included within the **main references section**, as these carry over from the original research within this proposal.

A.1 Introduction

Image captioning is a widely researched area, with many approaches, though it seems not many have applied it directly when investigating image object detection. There are many different image captioning models available, and after researching the topic area extensively, three of these have been selected for a deeper investigation within this project. These are Contrastive Language-Image Pretraining (CLIP) (Radford, et al., 2021), RegionCLIP (Zhong, et al., 2022), and unified Vision-Language Pre-training (VLP) (Zhou, et al., 2020). The main learning from these is that there are two prominent approaches to image captioning. These are Zero-Shot learning (ZSL) and Transfer Learning (TL) (among others). During this proposal, we delve into the details of these models and learning approaches within the Problem Domain section.

The main project objective is to develop an object detection process for a short video based on existing image captioning and description techniques, and there are many parts to consider while completing this project. The first objective is to select, train (if needed) and then analyse the captioning algorithms, picking the most suitable one for the project. The next objective will be to overcome the computational power limitations by using either keyframing or an alternative technique to reduce the number of images the algorithm needs to process. The final objective is to find a specific object within a short video within an appropriate timeframe and sensible GUI.

This project will involve developing an object detection process for short videos based on existing image captioning technologies, which is both interesting and challenging due to the various complexities involved. The challenges that need to be overcome are the training and analysis of the captioning algorithms, which will allow the most accurate algorithm to be selected. Another challenge is the computational power that will be available when running captioning algorithms on a short video, and this could be in the form of keyframing techniques or a workaround based on the concepts of binary searching. The main limitation of this project will be the length of the video that can be processed. A maximum duration of 2 minutes (equivalent to 3600 frames at 30 FPS) must be adhered to due to the limited computational power available. These challenges, along with the amount of current research in this area, make it a great project to demonstrate a wide array of knowledge within the subject of computer science.

A.2 Problem Domain

An image captioning model generally consists of two main parts: the visual detector (or image feature extractor) and the text generator (Sharma & Padha, 2023, p. 13620). The image feature extractor is responsible for processing the input image and extracting relevant objects like a car along with shapes, colours, and sizes. A Convolutional Neural Network (CNN) is usually used for this, and it produces context vectors, which are detailed summaries of visual elements within the image. The text generator then takes the content of an image (produced by the feature extractor) and translates it into a coherent, descriptive sentence.

A common approach that has been thoroughly researched is Contrastive Language-Image Pretraining (CLIP). CLIP is a neural network trained on text/image pairs, enabling it to predict the most relevant text snippet for any given image. An essential part of natural language is that it can describe an enormous set of concepts; the CLIP model is trained with this in mind and, therefore, can associate text with images (Radford, et al., 2021, p. 46). Existing work uses readily available smaller datasets with around 100,000 pairs, and a new dataset has been constructed using publicly available sources on the internet for CLIP with 400 million image-text pairs (Radford, et al., 2021, p. 2). CLIP does not require a specific training stage, meaning it can perform Zero-Shot classification. The names of all classes are used as a set of potential text pairings, which allows it to predict the most probable text-image pair and produce a result based on text descriptions (Radford, et al., 2021, p. 6).

RegionCLIP builds upon the original CLIP model discussed above, though it can caption specific regions instead of just captioning an entire image. (Zhong, et al., 2022, p. 16772) explains a ‘major domain shift’, in which the original CLIP algorithm was trained to caption an entire image. RegionCLIP is a significant extension of this functionality as it focuses on specific parts of the image rather than just understanding the image as a whole; this is known as fine-grained alignment (Zhong, et al., 2022, p. 16773). It creates “pseudo” labels for each smaller region, which are then combined with ground-truth text-image pairs (the correct caption for the image). The model is then trained on this, improving its ability to detect and describe objects. In essence, RegionCLIP enhances the original CLIP model by enabling it to zoom in and understand the finer details within an image, making it better for detailed image analysis.

The final model is a unified Vision-Language Pre-training (VLP) model. VLP is pre-trained on the Conceptual Captions dataset (Sharma, et al., 2018) consisting of 3 million text-image pairs. VLP is trained using two unsupervised learning objectives: Bidirectional Encoder Representations from Transformers (BERT) and sequence-to-sequence (seq2seq) (Zhou, et al., 2020). BERT is a language representation model (human language generator) designed

to pre-train by conditioning on the left and right context (Devlin, et al., 2019); this allows it to understand the context of a word based on all the words around it rather than just the words before it. Seq2seq involves using a specific self-attention mask which controls the context available to the language model during prediction (Zhou, et al., 2020, p. 4). VLP offers vast improvements in training speed and accuracy compared to older models due to its unified pre-training rather than traditional random or language-only pre-training.

Figure 1 provides a simplified overview of the image captioning techniques discussed above. The model will start by taking a source image and first extracting the features of that image, in this case, a tree, a red car, and a cloud. Then, it encodes these into a fixed-size vector, which is a fixed-length one-dimensional array of numbers representing the feature extracted. These vectors are then fed into a language model to generate the final captions.

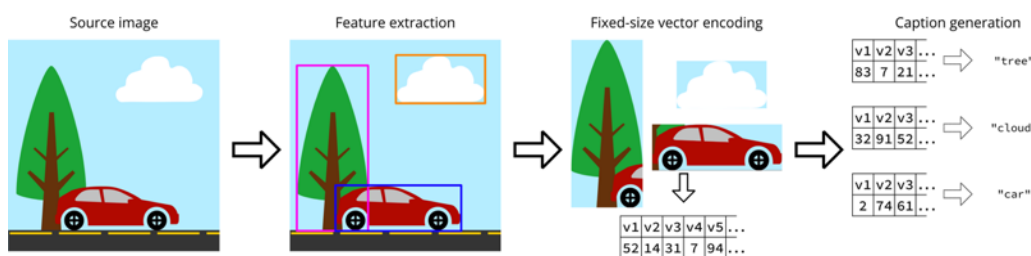


Figure 1: Overview of image captioning

These three models were selected because all three models (CLIP, RegionCLIP and VLP) can be used for zero-shot learning and transfer learning, meaning they can be easily compared with each other within the project. This also means that the same datasets and training can be used with them; thus, any results from testing will be accurate and relevant to the final algorithm selection. In addition, these models have also all been released within the past 5 years (2019, 2021 and 2022), which means they are relatively up-to-date and represent a broad range of approaches and accuracy ratings. A final added benefit is that they all have widely available codebases. Due to the time constraints of this project, efficiently applying and analysing these algorithms to achieve the end goal is much more important than recreating an algorithm that has already been created elsewhere.

The abovementioned algorithms allow for zero-shot (ZSL), which can form the feature extractor's basis. Zero-shot learning is a method to recognise and classify objects the algorithm hasn't seen before. It is completed using side information such as attributes (Xian, et al., 2019, p. 2251). During training, a ZSL model will collect semantic information, which involves extracting features from images and text, such as shapes, colours, words and phrases. Next, it maps the extracted features to a set of attributes, which in the context of ZSL are 'shared and nameable visual properties of objects' (Xian, et al., 2019, p. 2251); these are human-interpretable, for example: 'is red' or 'has legs'. Using seen classes (the predefined categories the model is trained on), the model can then learn the relationships

extracted, features extracted and the corresponding attributes. If the model is presented with an unseen class, it can use the learned mappings to predict the attributes from the features of the new data. Finally, the model will classify the unseen classes. This is done by using the same feature extraction method used initially and then matching these features with the semantic information collected during training.

Transfer learning (TL), conversely, is a method to recognise and classify objects that the algorithm has seen before. (Zhuang, et al., 2021, p. 44) states that 'a person who has learned the violin can learn the piano faster than others since both the violin and the piano are musical instruments and may share some common knowledge', a TL model works on this principle. Firstly, a pre-trained model needs to be selected, which has been trained on a large, pre-labelled dataset (such as ImageNet, discussed within the methodology section); this means that the pre-trained model will have already learned a vast number of features from an extensive dataset. This pre-trained model is then used as a feature extractor, processing new images and extracting their essential features. Layers can then be added on top of the pre-trained model, which allows the model to recognise specific objects related to the task given. These layers are then used to train the model further and will enable it to adapt to needs. The process of transfer learning effectively uses the pre-trained model to improve performance on the new task.

In addition, analysing the complexity of an image may be important when considering images we would use to test the suitability of the above algorithms for this project. A selection of images with varying complexity will be helpful in the completion of this project. This is due to the need to test and train the object detection algorithms above. We delve into more detail on this within the methodology section of this proposal.

This project aims to harness both the existing 'image-to-text' workflow, where we use image captioning algorithms and the 'text-to-image' workflow, where we query using text to find a corresponding image within the supplied video. Using these techniques, we aim to create software to find a specific object within a video. Based upon the different image description techniques discussed above, these could be used on a frame-by-frame basis within a video to detect the timestamp at which an object is fully visible on the screen. There are several challenges to this, which will be discussed in this proposal's methodology section.

A.3 Methodology

This methodology lays out the steps to complete the development of this project. It is broken down into sections that could be used as milestones to complete the development of this software.

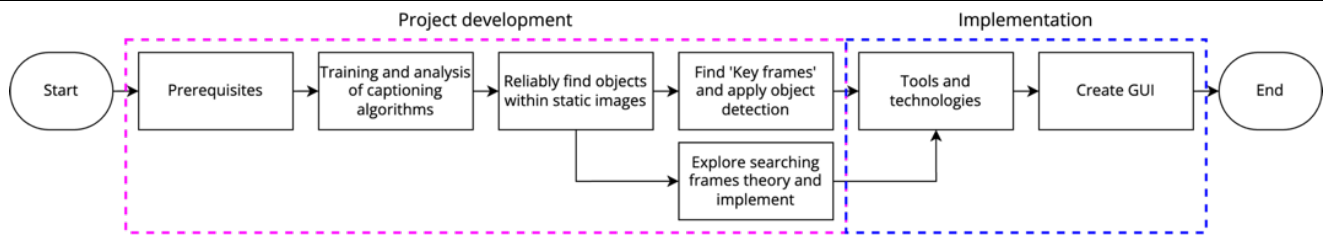


Figure 2: Flowchart of project methodology

A.4 Project Development

A.4.1 Prerequisites

Before diving into this project, a solid dataset must be found to train the algorithms. There are plenty to choose from, including millions of images varying in variety and quality. Some notable datasets include ImageNet (Deng, et al., 2009), COCO (Lin, et al., 2014), Visual Genome (Krishna, et al., 2016) and CIFAR-10/100 (Krizhevsky, 2009). The usability of these datasets would need to be analysed to determine how appropriate they are for developing this project.

Next, an appropriate set of images would need to be compiled. Due to the nature of this project, a variety of items should be considered, such as animals, objects, and video frames. Ultimately, the software will need to pick out objects from videos, which can have many variables such as occlusions, motion blur, and resolution (Sadkhan, et al., 2021). These images should vary in complexity, allowing for a thorough analysis of the algorithms.

A.4.2 Training and analysis of captioning algorithms

The image captioning algorithm may need to be trained, or in the case that the algorithm is pre-trained, it could be fine-tuned. There are many different datasets openly available online, some of which are mentioned above, and the pros and cons of each would need to be carefully analysed before deciding which of these to use for the project. (Zhou, et al., 2024) implies that the time it takes to train an algorithm directly correlates with the dataset size. This is a crucial point to consider, based on the compute power (GPU) available to train these algorithms. ImageNet, for example, provides different datasets with images of various resolutions, from the largest, including over fourteen million full-resolution photos, to the same set of images downsized to the following resolutions: 8x8, 16x16, 32x32, and 64x64.

Next, each object image captioning algorithm would need to be analysed in several ways to make sure the most appropriate algorithm is selected for the project. The following metrics could be used to evaluate the captions of the chosen algorithms: BLEU, ROUGE-L, CIDEr, and SPICE (Al-Shamayleh, et al., 2024). These work by measuring the similarities between the reference captions and the generated ones; while these may be useful to measure the

accuracy of the captions, other metrics will be necessary to measure the accuracy of object detection. A useful method to use would be the PASCAL VOC challenge. This consists of publicly available datasets with ground truth annotations along with a standardised evaluation software (Everingham, et al., 2015). There are several useful metrics in this paper, but the most useful one would be the Normalised Average Precision (NAP) metric, which allows performance comparisons of classification accuracy across classes (Everingham, et al., 2015).

Along with these more automated testing methods, manual testing should take place, and we can note the input and expected output using a set of images with well-defined and poorly defined objects. Manual testing using the compiled images from the prerequisites section will be incorporated in this way, and each image should be personally checked to see whether the algorithm has produced valuable results for the overall project. Figure 3 shows a brief example of what the expected input/output using an image with a streetlamp and a plane in the sky might be, which emphasises what needs to be analysed, but a measure of the accuracy needs to be automated as human biases and interpretations differ from person to person.



Figure 3: Plane and lamp captioning example

As mentioned in the opening paragraph of this section, the computing power available will be a crucial consideration in the captioning model selected. Computational efficiency will have to be measured, and training time is especially important as this would likely be the longest part of the process. In most cases, the accuracy of the algorithm directly impacts the speed at which it completes its task; therefore, a compromise between the speed and accuracy of the algorithm will need to be struck.

A.4.3 Reliably find objects within static images

This part can be summarised as creating a text input for the user to input text such as 'dog', 'person', or 'red car' and finding this within an image. There are many things to consider here before analysing the image. Firstly, the complexity of the text input can vary wildly. English has a wide array of ways to explain something, and the human mind can quickly understand these in many contexts when looking at an image. For example, to think about

a 'red car' is a straightforward concept for a person, but this could be described in a seemingly infinite number of ways: 'Red car', 'car which is red', and 'red passenger vehicle' could all mean the same thing to a person, but how can we create a computer program to understand what that means, thus applying it to an image captioning algorithm.

A.4.4 Find 'keyframes' and apply object detection

There are likely to be limitations to the computing power available. Therefore, a maximum of 2 minutes of video should be processed. Based on a 2-minute video at 30FPS, there would be a total of around 3600 frames, which would need to be analysed by the algorithms selected, which would require a vast amount of processing power.

A potential workaround to this that needs to be explored is keyframing, which is the process of retrieving the most important frames in a video. The goal of this would be to reduce the amount of data needed to find an object within the video. These keyframes are usually selected based on motion information, changes within a scene or the presence of objects in which we are interested (Sinulingga & Kong, 2023). It is often used in video surveillance (other sources indicate widespread use within digital animation, too) due to continuous monitoring being required.

A.4.5 Searching frames theory and implementation

Another workaround to the limitations of computing power could be to base it upon the simple theory of the binary searching algorithm. While this is merely a concept, it should be explored within this part of the methodology. If we were to take a set of frames from the maximum of 3600 frames we started with, for example, one frame after every 60 frames have passed (or every 2 seconds of video), this leaves us with a maximum total of 60 images to apply the detection algorithm to. First, the algorithm would split the video up into the frames mentioned above and would look for the object in its entirety. For example, in Figure 4, we take a very simple example of a red ball rolling across the screen from left to right. The first selected frame is blank, and as we skim through the video, we can see that the ball is fully visible within frame 120. The captioning algorithm will be able to understand this (i.e. returning something like 'red ball' as opposed to the previous frame being 'red semi-circle'), rewind to the halfway point between this frame and the last and move onto the 10th frame from there. This could be continued until the algorithm is scrubbing frame-by-frame. Depending on the speed of the captioning algorithm, this could be adjusted to find an approximation rather than an exact frame.

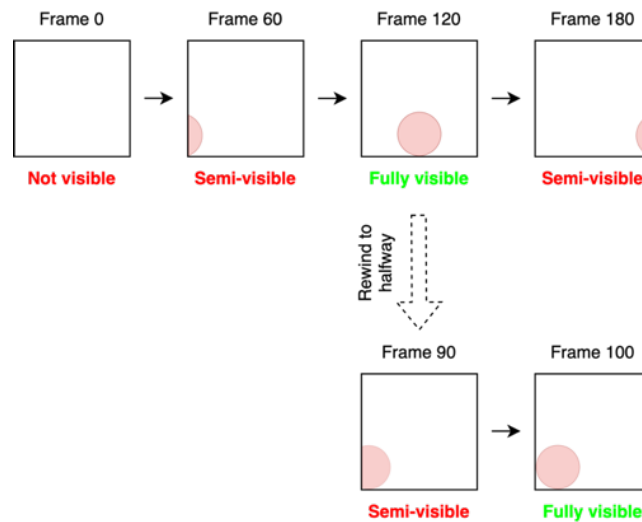


Figure 4: Example of 'searching algorithm' workaround

A.5 Implementation

A.5.1 Tools and Technologies

The first step will be to select the tools to complete this project. Based on virtually all research completed in this subject area, Python would be the programming language of choice used to complete this project, as most machine learning algorithms are created using Python.

Two notable machine learning frameworks are available in Python: PyTorch and TensorFlow. PyTorch is known for its ease of use due to its user-friendliness and good debugging tools; this means it's generally popular among researchers and academics as it can be used for prototyping and experimentation swiftly (Atreya, 2024). Conversely, TensorFlow has a steeper learning curve and is more suited for industry and production systems. For these reasons, PyTorch would be a better option due to its ease of use and user-friendly nature. Due to both frameworks being widely utilised, it may be necessary to use either or both in some capacity, depending on which image description techniques are used; that is to say, neither option would be invalid.

A.5.2 Graphical User Interface

Amalgamating all these elements into one sensible GUI would be challenging. The results would need to be returned in an appropriate manner. The GUI would be the user-facing part of this project; therefore, any algorithms would need to be pre-trained before running within any sort of GUI.

Several GUI elements would be needed, starting with a text input for the user to enter their prompt and then a file picker for selecting the video file in question. A preview window, which shows the current frame in which the software is currently processing would be

beneficial so that the user would know at what point and show progress of where the software has got to so far.

Figure 5 shows an example of what the GUI could look like with all the features implemented in the main view. This excludes any supplementary views, such as a final breakdown of time taken and the process of analysis.



Figure 5: Basic example GUI

A.6 Project Plan

Table 1 shows a complete breakdown of all tasks required to complete this project. While the list is exhaustive at this stage of the project, additional tasks will inevitably be added over time, as is common with software projects.

Task name	Description	Start date	End date
Create GUI	As part of prototyping within the contextual report, a Python GUI will be created with all the necessary pages for a complete project.	N/A	19/01/25
Finalize GUI	Any remaining GUI additions should be created here. Thus, the GUI should be complete, other than minor adjustments and placeholders being removed.	20/01/25	30/02/25
Dataset and image acquisition	Find a dataset to train algorithms and compile an appropriate set of testing images based on the project goal.	31/02/2025	09/02/25
Algorithm training and tuning	Train/fine-tune image captioning algorithms on selected datasets. This step includes exploring whether the datasets selected previously are appropriate.	10/02/25	16/02/25
Algorithm analysis	Analyse captioning algorithms based on single-image testing. Accuracy and quality are the most important metrics for this project.	17/02/25	02/03/25
Single-image captioning	Apply image captioning algorithm so it can be used within the project on one image.	03/03/25	09/03/25
Keyframing and binary	Explore keyframing methods and binary search algorithm theory as discussed in the methodology section	10/03/25	23/03/25

search theory			
Apply frame subsets	Apply the subset of frames to the captioning algorithm. This section overlaps the last, as this will be analysed for both methods	15/03/25	30/03/25
Quality evaluation	Evaluate the quality of captioning.	31/03/25	06/04/25
Accuracy evaluation	Evaluate the accuracy of object detection.	07/04/25	13/04/25
Full test	Thoroughly test every aspect of the application.	14/04/25	17/04/25

Table 1: Task breakdown

Figure 6 is a Gantt chart that displays an overview of the project, week by week. The chart assumes a GUI prototype has been created (as this will form part of the contextual report) and can be cross-referenced with Table 1 for more in-depth descriptions of each task.

Task	Week 1 20/01/25	Week 2 27/01/25	Week 3 03/02/25	Week 4 10/02/25	Week 5 17/02/25	Week 6 24/02/25	Week 7 03/03/25	Week 8 10/03/25	Week 9 17/03/25	Week 10 24/03/25	Week 11 31/03/25	Week 12 07/04/25	Week 13 14/04/25
Finalize GUI													
Dataset and image acquisition													
Algorithm training and tuning													
Algorithm analysis													
Single-image captioning													
Key framing and binary search theory													
Apply frame subsets													
Quality evaluation													
Accuracy evaluation													
Full test													

Figure 6: Gantt chart of project

A.7 Evaluation

The overall goal of this project is to take the current 'image-to-text' workflow and create software that can find a specific object within a video. After analysis to find the best image captioning software based on the methods selected, this can be run on a frame-by-frame basis, and the software will be able to detect the timestamp at which an object is fully visible on the screen.

This project will be deemed successful if the overall goal mentioned above is met. This project should accurately be able to retrieve the timestamp in which an object is fully visible within the frame from a 2-minute video (around 3600 frames of video assuming 30FPS). Due to the limitations of computing power available, the software should be able to complete the task within 30 minutes (excluding model training if needed). From a user perspective, given how complex the task is, this is a reasonable timeframe.

There are several points within the methodology of this project which will need to be evaluated to measure the overall success of this project. Firstly, the model selected for use within the final implementation will be thoroughly evaluated in terms of the quality of the description generated by the algorithm and the accuracy of the object detected. Next, the

way objects are discovered will need to be analysed. Keyframing has been discussed along with an alternate theory as to how to find an object; this will need to be analysed both on the speed and effectiveness of the approach. Finally, after these approaches have been analysed, the project will be amalgamated into the sensible GUI discussed, completing the project successfully.

Appendix B – Extended Results

B.1 Image captioning

B.1.1 Automated script (third-party data)

Note: The full results file contains well over 1 million lines of CSV data (~600MB), there for the first 3 sets have been included in this section. The full data is available within the GitHub repo linked on the title page, and in Appendix F. The reference images are available in Appendix C.3, the first three items correspond directly to these results.

Timestamp	Model	Reference(s)	Candidate	Metric	Score
1743255434	OpenCLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a close up of a bottle of listerine	BLEU	0.1652
1743255436	OpenCLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a close up of a bottle of listerine	METEOR	0.3681
1743255436	OpenCLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a close up of a bottle of listerine	ROUGE-1	0.2978
1743255436	OpenCLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a close up of a bottle of listerine	ROUGE-2	0.1042

		'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']			
1743255436	OpenCLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a close up of a bottle of listerine	ROUGE-L	0.2074
1743255437	BLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a bunch of bottles of listerine on a shelf	BLEU	0.3064
1743255437	BLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a bunch of bottles of listerine on a shelf	METEOR	0.5453
1743255437	BLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a bunch of bottles of listerine on a shelf	ROUGE-1	0.366
1743255437	BLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a bunch of bottles of listerine on a shelf	ROUGE-2	0.1672

		'Listerine zero bottles which is less intense and zero alcohol.']			
1743255437	BLIP	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	a bunch of bottles of listerine on a shelf	ROUGE-L	0.3228
1743255438	Florence2	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	The image shows six bottles of Listerine Zero mouthwash on a shelf. The bottles are blue in color and have a white label with the brand name "Listerine" written in bold black letters. The label also has an image	BLEU	0.1224
1743255438	Florence2	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	The image shows six bottles of Listerine Zero mouthwash on a shelf. The bottles are blue in color and have a white label with the brand name "Listerine" written in bold black letters. The label also has an image	METEOR	0.4259
1743255438	Florence2	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	The image shows six bottles of Listerine Zero mouthwash on a shelf. The bottles are blue in color and have a white label with the brand name "Listerine" written in bold black letters. The label also has an image	ROUGE-1	0.2229
1743255438	Florence2	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	The image shows six bottles of Listerine Zero mouthwash on a shelf. The bottles are blue in color and have a white label with the brand name "Listerine" written	ROUGE-2	0.1203

		'Listerine zero bottles which is less intense and zero alcohol.']	in bold black letters. The label also has an image		
1743255438	Florence2	['Five Listerine Zero mouthwash bottles on a store shelf', 'Many bottles of Listerine Zero are placed together.', 'Listerine Zero mouthwash is less intense with zero alcohol.', 'Several bottles of Listerine Zero sit on display on a shelf', 'Listerine zero bottles which is less intense and zero alcohol.']	The image shows six bottles of Listerine Zero mouthwash on a shelf. The bottles are blue in color and have a white label with the brand name "Listerine" written in bold black letters. The label also has an image	ROUGE-L	0.2145
1743255439	OpenCLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a white clock on a wall with roman numerals .	BLEU	0.193
1743255439	OpenCLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a white clock on a wall with roman numerals .	METEOR	0.378
1743255439	OpenCLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a white clock on a wall with roman numerals .	ROUGE-1	0.3081
1743255439	OpenCLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which	a white clock on a wall with roman numerals .	ROUGE-2	0.0688

		say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']			
1743255439	OpenCLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a white clock on a wall with roman numerals .	ROUGE-L	0.2655
1743255439	BLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a clock hanging on the wall of a building	BLEU	0.2167
1743255439	BLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a clock hanging on the wall of a building	METEOR	0.2184
1743255439	BLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a clock hanging on the wall of a building	ROUGE-1	0.2622
1743255439	BLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a clock hanging on the wall of a building	ROUGE-2	0.0461

		front.', 'A clock says Angel Square London on its face.']			
1743255439	BLIP	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	a clock hanging on the wall of a building	ROUGE-L	0.2229
1743255440	Florence2	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	The image is a photograph of a round clock hanging on a white wall. The clock has a white face with black Roman numerals and a black hour hand. The hour hand is pointing to the 12 o'clock position. Below the hour	BLEU	0.0431
1743255440	Florence2	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	The image is a photograph of a round clock hanging on a white wall. The clock has a white face with black Roman numerals and a black hour hand. The hour hand is pointing to the 12 o'clock position. Below the hour	METEOR	0.2843
1743255440	Florence2	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	The image is a photograph of a round clock hanging on a white wall. The clock has a white face with black Roman numerals and a black hour hand. The hour hand is pointing to the 12 o'clock position. Below the hour	ROUGE-1	0.1638
1743255440	Florence2	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the	The image is a photograph of a round clock hanging on a white wall. The clock has a white face with black Roman numerals and a black hour hand. The hour hand is pointing to	ROUGE-2	0.0157

		front.', 'A clock says Angel Square London on its face.']	the 12 o'clock position. Below the hour		
1743255440	Florence2	['The time is now 10:22 according to this clock.', 'A large clock by Harris and Thompson displays a time of 10:22.', 'Large roman numeral clock by Angel Square London.', 'Clock with roman numeral numbers which say "Angel Square" on the front.', 'A clock says Angel Square London on its face.']	The image is a photograph of a round clock hanging on a white wall. The clock has a white face with black Roman numerals and a black hour hand. The hour hand is pointing to the 12 o'clock position. Below the hour	ROUGE-L	0.1494
1743255441	OpenCLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a website with a phone on it .	BLEU	0.1836
1743255441	OpenCLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a website with a phone on it .	METEOR	0.3757
1743255441	OpenCLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a website with a phone on it .	ROUGE-1	0.1754
1743255441	OpenCLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a website with a phone on it .	ROUGE-2	0.0426

1743255441	OpenCLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a website with a phone on it .	ROUGE-L	0.1754
1743255441	BLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a cell phone	BLEU	0.2191
1743255441	BLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a cell phone	METEOR	0.2222
1743255441	BLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a cell phone	ROUGE-1	0.1384
1743255441	BLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	a screen shot of a cell phone	ROUGE-2	0.0235
1743255441	BLIP	['Lloyd Budd's email service is powered by Google.', 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with	a screen shot of a cell phone	ROUGE-L	0.1384

		Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']			
1743255443	Florence2	["Lloyd Budd's email service is powered by Google.", 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	The image is a screenshot of the homepage of the Google Gmail website. The page is titled "Welcome to Lloyd Budd" and has a message that reads "Sign in to your account at Lloyd Budd". Below the message, there is an image	BLEU	0.0408
1743255443	Florence2	["Lloyd Budd's email service is powered by Google.", 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	The image is a screenshot of the homepage of the Google Gmail website. The page is titled "Welcome to Lloyd Budd" and has a message that reads "Sign in to your account at Lloyd Budd". Below the message, there is an image	METEOR	0.2273
1743255443	Florence2	["Lloyd Budd's email service is powered by Google.", 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	The image is a screenshot of the homepage of the Google Gmail website. The page is titled "Welcome to Lloyd Budd" and has a message that reads "Sign in to your account at Lloyd Budd". Below the message, there is an image	ROUGE-1	0.1929
1743255443	Florence2	["Lloyd Budd's email service is powered by Google.", 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	The image is a screenshot of the homepage of the Google Gmail website. The page is titled "Welcome to Lloyd Budd" and has a message that reads "Sign in to your account at Lloyd Budd". Below the message, there is an image	ROUGE-2	0.0239
1743255443	Florence2	["Lloyd Budd's email service is powered by Google.", 'A Gmail screen shot welcoming Lloyd Budd asking for Username and Password.', 'A Gmail account is connected to something called Lloyd Budd.', 'a mail page with	The image is a screenshot of the homepage of the Google Gmail website. The page is titled "Welcome to Lloyd Budd" and has a message that reads "Sign in to your account at Lloyd Budd".	ROUGE-L	0.1695

		Lloyd Bud on it', 'A fire fox login page to lloy budd gmail.']	Below the message, there is an image		
--	--	--	--------------------------------------	--	--

B.1.2 Manual script (curated data)

Note: Reference the file name with Appendix C – Manually collected testing images. Only the second run of three for each model is included in this section, due to most of the data being the same or similar, it was not necessary to include all 180 lines of data here. The full data is available within the GitHub repo mentioned on the title page, and in Appendix F.

Model	File name	Caption	Time taken (s)
OpenCLIP	sp_trucks.jpg	a bunch of trucks on a highway with a lot of traffic .	2.083
OpenCLIP	ducks.jpeg	a couple of birds standing on top of a grass covered field .	2.066
OpenCLIP	dunes.jpeg	the sun is setting over the ocean on the beach .	1.8
OpenCLIP	plane_lamp.jpeg	an airplane is flying in a cloudy blue sky .	1.63
OpenCLIP	black_bird.jpeg	a black bird standing next to a potted plant .	1.688
OpenCLIP	dog_1.jpeg	a small dog standing on top of a pile of rocks .	1.889
OpenCLIP	sp_highway_bridge.jpg	cars are driving on a highway near a body of water .	1.971
OpenCLIP	geese_2.jpeg	a flock of geese crossing the street in the rain .	1.779
OpenCLIP	cat_1.jpeg	a cat sitting on top of a stone floor .	1.687
OpenCLIP	cat_2.jpeg	a cat sitting on the ground looking at the camera .	1.792
OpenCLIP	geese_1.jpeg	a group of geese walking across a wet road .	1.716
OpenCLIP	dog_2.jpeg	a close up of a dog 's face with a white background .	1.98
OpenCLIP	sp_running_race.jpg	a large group of people running in a race .	1.647
OpenCLIP	sp_high_street_people.jpg	a group of people walking down a street .	1.549
OpenCLIP	sp_highway.jpg	a bunch of cars that are on a highway .	1.773
OpenCLIP	sp_red_car_autumn.jpg	an old red car is driving down the road .	1.72
OpenCLIP	sp_athletics_water.jpg	a group of people running in a pool of water .	1.759
OpenCLIP	sp_horses_splashing.jpg	a group of white horses running through the water .	1.753
OpenCLIP	sp_night_street_people_cars.jpg	two people walking down a street at night .	1.623
OpenCLIP	sp_people_walking_blur.jpg	a crowd of people crossing a street on a crosswalk .	1.866
SalesForceBLIP	sp_trucks.jpg	a long highway	0.537
SalesForceBLIP	ducks.jpeg	a group of ducks walking in a park	0.577
SalesForceBLIP	dunes.jpeg	a beach with grass and a sunset in the background	0.611
SalesForceBLIP	plane_lamp.jpeg	a plane flying over a statue in a park	0.525
SalesForceBLIP	black_bird.jpeg	a bird is sitting on a tree stump	0.548
SalesForceBLIP	dog_1.jpeg	a small dog standing on a rocky area	0.548
SalesForceBLIP	sp_highway_bridge.jpg	a city street with a lot of traffic	0.607
SalesForceBLIP	geese_2.jpeg	a group of geese walking in the rain	0.527
SalesForceBLIP	cat_1.jpeg	a cat walking on a stone walkway	0.498
SalesForceBLIP	cat_2.jpeg	a cat is sitting on a stone floor	0.547
SalesForceBLIP	geese_1.jpeg	a group of geese walking in the rain	0.549

SalesForceBLIP	dog_2.jpeg	a small dog sitting on a table	0.475
SalesForceBLIP	sp_running_race.jpg	a group of people running in a race	0.457
SalesForceBLIP	sp_high_street_people.jpg	a group of people walking down a street	0.579
SalesForceBLIP	sp_highway.jpg	a long highway	0.485
SalesForceBLIP	sp_red_car_autumn.jpg	a red car parked in the middle of a forest	0.517
SalesForceBLIP	sp_athletics_water.jpg	a group of people running in a race	0.518
SalesForceBLIP	sp_horses_splashing.jpg	two horses running in the water	0.541
SalesForceBLIP	sp_night_street_people_cars.jpg	a couple walking down a street at night	0.673
SalesForceBLIP	sp_people_walking_blur.jpg	a crowd of people crossing a street	0.478
Florence2	sp_trucks.jpg	The image shows a highway with multiple lanes of traffic. There are several trucks and cars on the road, including a red truck, a white truck, and a green truck. The highway is divided into two sections, with a toll booth on the left side and a bridge on the right side. In the background, there are trees and hills, and the sky is cloudy. The image appears to be taken from a high angle, looking down on the highway.	10.952
Florence2	ducks.jpeg	The image shows two geese walking on a grassy field with fallen leaves scattered around them. The geese are facing towards the right side of the image and appear to be walking side by side. The grass is green and there are a few brown leaves scattered on the ground. The sky is blue and the weather appears to be sunny and clear.	5.922
Florence2	dunes.jpeg	The image is a photograph of a beautiful sunset over the ocean. The sky is a mix of orange, yellow, and blue hues, with the sun setting in the background. The sun is partially hidden behind the horizon, casting a warm glow over the water. In the foreground, there are tall grasses swaying in the wind. The grasses are silhouetted against the sky, creating a contrast of light and shadow. The ocean is visible in the distance, with small waves crashing onto the shore. The overall mood of the image is peaceful and serene.	6.754
Florence2	plane_lamp.jpeg	The image shows an airplane flying in the sky. The sky is a bright blue with a few white clouds scattered across it. The airplane is in the center of the image, with its wings spread wide and its nose pointing towards the right side of the frame. The clouds are white and fluffy, and they appear to be floating in the air. In the foreground, there is a black lamp post with a round top and a small statue on top. The lamp post is silhouetted against the sky, and the background is a clear blue sky.	5.538
Florence2	black_bird.jpeg	The image shows a black bird perched on a wooden post in a garden. The bird is facing towards the right side of the image and appears to be looking towards the left side. It has a yellow beak and is perched on the post with its wings slightly spread. In the background, there is a large green plant with long, pointed leaves and red flowers. The garden is filled	6.287

		with various plants and shrubs, and there are a few rocks scattered around.	
Florence2	dog_1.jpeg	The image shows a small black and white dog standing on a rocky ground. The dog is wearing a leash and appears to be looking towards the right side of the image. The ground is covered in small rocks and pebbles, and there are a few small plants scattered around. On the ground next to the dog, there is a pair of blue shoes. The shoes appear to be new and unused.	5.371
Florence2	sp_highway_bridge.jpg	The image shows a long, straight road with multiple lanes of traffic. The road is divided into two sections, with the left side of the road being a river and the right side being a cityscape. The river is visible in the background, with buildings and trees on both sides. The sky is overcast and the overall mood of the image is gloomy. There are several red buses and cars on the road, indicating that it is a busy highway. The image appears to be taken from a high vantage point, looking down on the highway.	6.066
Florence2	geese_2.jpeg	The image shows a group of geese walking on a wet road. The road is surrounded by a grassy field and there are trees and power lines in the background. The sky is overcast and the ground is wet, indicating that it has recently rained. The geese are walking in a line, with some walking towards the right side of the image and others following closely behind. They are all facing the same direction and appear to be walking in different directions.	5.045
Florence2	cat_1.jpeg	The image shows a cat sitting on a large, rectangular stone pavement. The pavement appears to be old and weathered, with cracks and crevices visible on the surface. The cat is brown and black in color and is looking towards the right side of the image. It is standing on all fours, with its front paws resting on the ground and its tail curled around its body. The background is blurred, but it seems to be an outdoor setting with a brick wall visible in the top left corner.	5.253
Florence2	cat_2.jpeg	The image shows a cat sitting on a large, rectangular stone pavement. The pavement appears to be old and weathered, with cracks and crevices visible on the surface. The cat is facing towards the right side of the image, with its head turned towards the left side. It is sitting on the pavement with its front paws resting on the ground and its tail curled around its body. Its fur is a mix of brown and black, and its eyes are dark and alert. The background is blurred, but it seems to be an outdoor setting with trees and other vegetation visible.	5.967
Florence2	geese_1.jpeg	The image shows a group of geese walking on a wet road. The road is surrounded by a grassy field and there are trees and power lines in the background. The sky is overcast and the ground is wet, indicating	6.168

		that it has recently rained. The geese are walking in a line, with some walking towards the right side of the image and others following closely behind. They are all facing the same direction and appear to be walking in different directions.	
Florence2	dog_2.jpeg	The image is a close-up of a small dog's face. The dog appears to be a miniature schnauzer, with black and white fur on its head and body. Its eyes are dark and its ears are perked up, giving it an alert and curious expression. Its nose is black and its mouth is slightly open, as if it is looking directly at the camera. The background is a plain white wall, and there is a door visible on the left side of the image.	5.973
Florence2	sp_running_race.jpg	The image shows a large group of people running in a marathon. The runners are of different ages and genders, and they are all wearing athletic clothing. The road is lined with trees and there is a building in the background. The sky is blue and the sun is shining, indicating that it is a bright and sunny day. The people are running closely together, and some are wearing backpacks and running shoes. The overall mood of the image is energetic and determined.	5.439
Florence2	sp_high_street_people.jpg	The image shows a busy street in a European city. The street is lined with tall buildings on both sides, with shops and restaurants visible on the left and right sides. The buildings are painted in a variety of colors, including red, orange, and white, and there are signs and banners hanging from the buildings. People are walking on the street, some carrying bags, while others are riding bicycles. The sky is blue and the weather appears to be sunny and warm. The overall atmosphere of the image is bustling and bustling.	5.401
Florence2	sp_highway.jpg	The image shows a busy highway with multiple lanes of traffic. There are multiple cars of different models and colors driving on the road, including white, silver, and black. The cars are all facing the same direction and appear to be moving at a high speed. The road is divided into two sections, with the left side of the image showing a curved curve and the right side showing a straight line. The sky is clear and blue, indicating that the photo was taken from a high angle. The image appears to be taken from an elevated position, looking down on the highway.	5.967
Florence2	sp_red_car_autumn.jpg	The image shows a red vintage car parked on the side of a road in a forest. The car is facing towards the right side of the image and is surrounded by tall trees on both sides. The trees have green leaves and the ground is covered in fallen leaves. The sky is visible through the trees and the road appears to be wet, suggesting that it has recently rained. On the back of the car, there is a brown suitcase hanging from the roof. The license plate reads "95-59".	5.265

Florence2	sp_athletics_water.jpg	The image shows a group of athletes competing in a swimming pool. The pool is blue and there are orange cones around the edges. In the background, there is a large stadium with rows of empty seats. The athletes are in the middle of a race, with one athlete jumping over a hurdle. The athlete on the left is wearing a red and white uniform, while the athlete in the green and yellow uniform is running towards the finish line. The other athletes are wearing blue and green uniforms. The sky is cloudy and the water in the pool is splashing around the athletes.	5.833
Florence2	sp_horses_splashing.jpg	The image shows four white horses running through the water. The horses are galloping through the shallow water, splashing around and creating a splash as they run. The sky is clear and blue, and the water is a deep blue-green color. The sun is shining brightly, casting a warm glow on the horses and the surrounding landscape. The horse in the foreground is in the center of the image, with its head turned to the side and its mane and tail flowing in the wind. The other three horses are following closely behind, with their heads held high and their manes and tails flowing behind them. They appear to be in motion, as if they are running at a fast pace.	7.529
Florence2	sp_night_street_people_cars.jpg	The image shows a street at night in a European city. The street is lined with old stone buildings on both sides, with shops and restaurants on the right side. The buildings are lit up with street lamps, creating a warm glow. In the center of the street, there are several people walking on the sidewalk, some of them carrying bags. On the left side of the image, there is a large stone building with a large arched doorway. The sky is dark, indicating that it is nighttime. There are also several cars parked along the street and a few people walking around. The overall atmosphere of the scene is bustling and bustling.	6.882
Florence2	sp_people_walking_blur.jpg	The image is a close-up of a busy street scene with a large group of people walking across a zebra crossing. The people are of different ages, genders, and ethnicities, and they are all wearing casual clothes. The image is taken from a high angle, looking down on the people as they cross the street. The background is blurred, but it appears to be a busy city street with buildings and trees. The overall mood of the image is chaotic and bustling.	5.922

B.2 Unit testing

Note: The results were originally presented via a PyCharm HTML export, these were then converted to a CSV using a Python script, and the following table contains all results from all unit tests created.

Suite	Test	P/F	Time taken (ms)
TestCompletionHandler	test_generate_completion_report	P	16
TestCompletionHandler	test_generate_pdf_content	P	2
TestCompletionHandler	test_generate_pdf_content_with_missing_images	P	1
TestCompletionHandler	test_init	P	1
TestCompletionHandler	test_init_with_empty_matching_frames	P	3
TestCompletionHandler	test_init_with_invalid_matching_frames	P	1
TestCompletionHandler	test_save_pdf	P	2
TestCompletionHandler	test_setup_pdf	P	1
TestCompletionHandler	test_setup_pdf_with_invalid_path	P	0
TestFileHandler	test_init	P	3
TestFileHandler	test_init_with_invalid_file_path	P	4
TestFileHandler	test_process_keyframes	P	20080
TestFileHandler	test_process_keyframes_with_empty_directory	P	9
TestFileHandler	test_retrieve_keyframes	P	8
TestFileHandler	test_retrieve_keyframes_with_exception	P	4
TestFileHandler	test_save_caption	P	5
TestFileHandler	test_save_caption_with_none_values	P	3
TestFileHandler	test_setup_caption_file	P	5
TestFrameDisplayer	test_clear_frame	P	3
TestFrameDisplayer	test_display_frame	P	13
TestFrameDisplayer	test_display_frame_with_invalid_image	P	2
TestFrameDisplayer	test_display_frame_with_nonexistent_file	P	1
TestFrameDisplayer	test_init	P	1
TestFrameDisplayer	test_init_with_none_preview_element	P	1
TestGlobalTools	test_clear_frame_directories	P	1
TestGlobalTools	test_clear_frame_directories_with_non_files	P	1
TestGlobalTools	test_clear_frame_directories_with_permission_error	P	0
TestGlobalTools	test_create_directories	P	1
TestGlobalTools	test_create_directories_with_permission_error	P	1
TestImageCaptioningHandler	test_frame_caption	P	1
TestImageCaptioningHandler	test_frame_caption_with_empty_frame_name	P	2
TestImageCaptioningHandler	test_get_caption	P	1
TestImageCaptioningHandler	test_get_caption_with_invalid_image	P	108000
TestImageCaptioningHandler	test_get_caption_with_nonexistent_file	P	1
TestImageCaptioningHandler	test_init	P	3
TestImageCaptioningHandler	test_init_with_invalid_output_dir	P	1
TestKeyFrameHandler	test_calculate_frame_difference	P	3
TestKeyFrameHandler	test_calculate_threshold	P	2
TestKeyFrameHandler	test_extract_keyframes	P	3
TestKeyFrameHandler	test_generate_histograms	P	2

TestKeyFrameHandler	test_init	P	3
TestKeyFrameHandler	test_init_with_invalid_parameters	P	2
TestKeyFrameHandler	test_split_video	P	313
TestKeyFrameHandler	test_split_video_with_invalid_video	P	5
TestKeyFrameHandler	test_split_video_with_read_failure	P	4
TestLanguageHandler	test_compare_caption_to_search_term_with_match	P	1410
TestLanguageHandler	test_compare_caption_to_search_term_with_punctuation	P	0
TestLanguageHandler	test_compare_caption_to_search_term_without_match	P	0
TestLanguageHandler	test_compare_caption_with_irrelevant_words_only	P	0
TestLanguageHandler	test_compare_caption_with_multiple_matches	P	0
TestLanguageHandler	test_compare_caption_with_none_caption	P	0
TestLanguageHandler	test_compare_caption_with_quoted_text	P	1
TestLanguageHandler	test_compare_caption_with_special_characters_only	P	0
TestLanguageHandler	test_compare_caption_with_very_large_caption	P	2
TestLanguageHandler	test_get_search_synonyms	P	0
TestLanguageHandler	test_get_search_synonyms_with_wordnet_failure	P	0
TestLanguageHandler	test_init	P	0
TestLanguageHandler	test_init_with_blank_search_term	P	0
TestLanguageHandler	test_real_wordnet_lookups	P	29
TestLanguageHandler	test_remove_irrelevant_words	P	0
TestLanguageHandler	test_remove_irrelevant_words_with_non_set_input	P	0
TestLanguageHandler	test_split_caption	P	0
TestLanguageHandler	test_split_caption_with_non_string_input	P	0
TestLanguageHandler	test_wordnet_returns_no_synsets	P	0

B.3 Manual testing

B.3.1 Feature testing

Note: The following results comes from a manually created Microsoft Excel spreadsheet of each feature. For space, the final three columns are shortened to ‘1’, ‘2’, ‘3’ = ‘Run 1’, ‘Run 2’, ‘Run 3’. The same applies in column 1: ‘Error’ = ‘Erroneous’.

Data type	Test objective	Input	Expected output	1	2	3
Normal	File import	.mp4 file	Video import	P	P	P
Error	File import	.pdf file	Does not allow import	P	P	P
Normal	Short video (2 frames) import	two_frames.mp4	Video import	P	P	P
Normal	Medium video (2 seconds/60 frames) import	turtle_2s.mp4	Video import	P	P	P
Normal	Long video (1+ minutes) import	mixed_scenes_2m.mp4	Video import	P	P	P

Error	Corrupt video input	broken_video.mp4	File cannot be opened error	F	P	P
Normal	Single word search term (turtle_2s.mp4)	turtle	Find in video	P	P	P
Error	Single word search term (turtle_2s.mp4)	tree	Not find in video	P	P	P
Normal	Single word search term (unfocused_motorway_2s.mp4)	truck	Find in video	P	P	P
Error	Multiple word search term (unfocused_motorway_2s.mp4)	white truck	Not find in video	P	P	P
Normal	Single word synonym (unfocused_motorway_2s.mp4)	motortruck	Find in video	P	P	P
Error	Multiple word synonym (unfocused_motorway_2s.mp4)	white motortruck	Not find in video	P	P	P
Normal	PDF output on successful find	two_frames.mp4	Found 'turtle' PDF output	P	P	P
Error	PDF output on unsuccessful find	two_frames.mp4	No PDF option	P	P	P
Normal	Deletion of temporary files on open	N/A	No frames/files in any internal dir (except logs)	P	P	P
Normal	Data logging	two_frames.mp4	Log of console output in log dir	P	P	P

B.3.2 Performance testing

Note: Open Broadcasting Software (OBS) was used to record and time each test, as it generally only added around 1-2% load to the CPU meaning that unattended testing was reasonable. The second table in this section includes permanent unlisted YouTube links for unedited recorded runs. For each image found in a video, 15 seconds is removed from the total time to account for intentional processing pauses.

KFR (%) = Ratio of key frames to original video frames count as a percentage

Video	Contents	Run 1			Run 2			Run 3		
		Phras e	Tim e	KFR (%)	Phras e	Tim e	KFR (%)	Phrase	Tim e	KFR (%)
four_frames.mp4	Town square Turtle Turtle Town square	turtle	18 s	50%	clock	19 s	50%	buildin g	15 s	50%
turtle_30s.mp4	Turtle swimming in sea	turtle	1 h 23 m 1 s	20.9 %	-	-		-	-	
mixed_scenes_20s.mp4	Bird on post	turtle	29 m 68 s	6.6%	truck	22 m 1 s	6.6 %	feeder	19 m 39 s	6.6 %

	Blurred video of market stall Blurred truck leaving depot Dashcam driving Flamingos Bird eating seed Turtle Market Blurred road CCTV from above									
mixed_scenes_2m.mp4 4	Dashcam driving Blurred video of market stall Bird on post Highway CCTV Turtle Blurred road CCTV from above	turtle	3 h 7 m 33 s DNF	-	-	-		-	-	

Testing Video	Run 1	Run 2	Run 3
four_frames.mp4 [See table below]	youtu.be/Zgnmh5KTBhk	youtu.be/aN0bUcMgcjI	youtu.be/SUc6Pj6yqzo
turtle_30s.mp4 youtu.be/DPgkxnkShlo	youtu.be/nHuh2qffcQI	-	-
mixed_scenes_20s.mp4 youtu.be/Jg8ktPI182M	youtu.be/xjVjl_bptCE	youtu.be/CQgS1dwYQRw	youtu.be/tHGOI39cl-Y
mixed_scenes_2m.mp4 youtu.be/ZO3OLh2WsNg	youtu.be/Gm5k1z6DX4I	-	-

four_frames.mp4			
Frame 1	Frame 2	Frame 3	Frame 4



B.4 PDF report output

B.4.1 Turtle sample

Note: Four pages are selected at random points in this 188-page PDF report, to demonstrate the issue of searching a simple video.

Report for turtle_30s

Search term: turtle

Matching Frame: 0002_67.jpg

Caption	Timestamp	Frame Number
The image is an underwater photograph of a sea turtle swimming over a coral reef. The turtle is in the center of the image, with its head turned towards the right side of the frame. It has a brown shell with white spots on its head and back, and its flippers are spread out as it moves through the water. The ocean floor is covered in small rocks and coral, and there are a few small fish visible in the background. The water is a deep blue color, and the overall scene is peaceful and serene.	00:00:067	0002

Report for turtle_30s

Search term: turtle

Matching Frame: 0295_9833.jpg

Caption	Timestamp	Frame Number
The image is an underwater photograph of a sea turtle swimming over a coral reef. The turtle is facing towards the right side of the image and its head is turned towards the left side. It has a brown and white shell with a pattern of small circles and lines on its head. The sea turtle is swimming over the coral reef, which is covered in small rocks and coral. The water is clear and blue, and there are a few small fish visible in the background. The overall scene is peaceful and serene.	00:09:833	0295

Report for turtle_30s

Search term: turtle

Matching Frame: 0485_16167.jpg

Caption	Timestamp	Frame Number
The image is an underwater photograph of a sea turtle swimming in the ocean. The turtle is facing towards the right side of the image and its head is turned towards the left side. It has a brown shell with black spots on its head and neck. Its flippers are spread out as it moves through the water. The ocean floor is covered in small rocks and coral reefs. The water is a deep blue color and there are a few small fish visible in the background. The overall scene is peaceful and serene.	00:16:167	0485

Report for turtle_30s

Search term: turtle

Matching Frame: 0884_29467.jpg

Caption	Timestamp	Frame Number
The image is an underwater photograph of a sea turtle swimming in the ocean. The turtle is in the center of the image, with its head above the water and its body partially submerged in the water. It has a red and white striped shell and a black head and neck. The water is a deep blue-green color and there are small rocks and coral reefs visible below the surface. The sun is shining brightly, creating a beautiful reflection on the water's surface.	00:29:466	0884

B.4.2 Mixed scenes sample

Note: Taken from separate runs

Report for mixed_scenes_20s

Search term: feeder

Matching Frame: 0429_14300.jpg

A close-up photograph of a small, rectangular metal bird feeder. The feeder is attached to a tree trunk with a black ribbon. It has a flat top and a small opening on the side. Inside the feeder, there are small, light-colored seeds. The background is a blurred green, suggesting a wooded area.

Caption	Timestamp	Frame Number
The image shows a small bird feeder hanging from the side of a tree trunk. The feeder is made of metal and has a rectangular shape with a flat top. It is secured to the tree with a black ribbon. Inside the feeder, there are small white seeds, which appear to be peanuts. The background is blurred, but it appears to be a wooded area with green foliage.	00:14:300	0429

Report for mixed_scenes_20s

Search term: truck

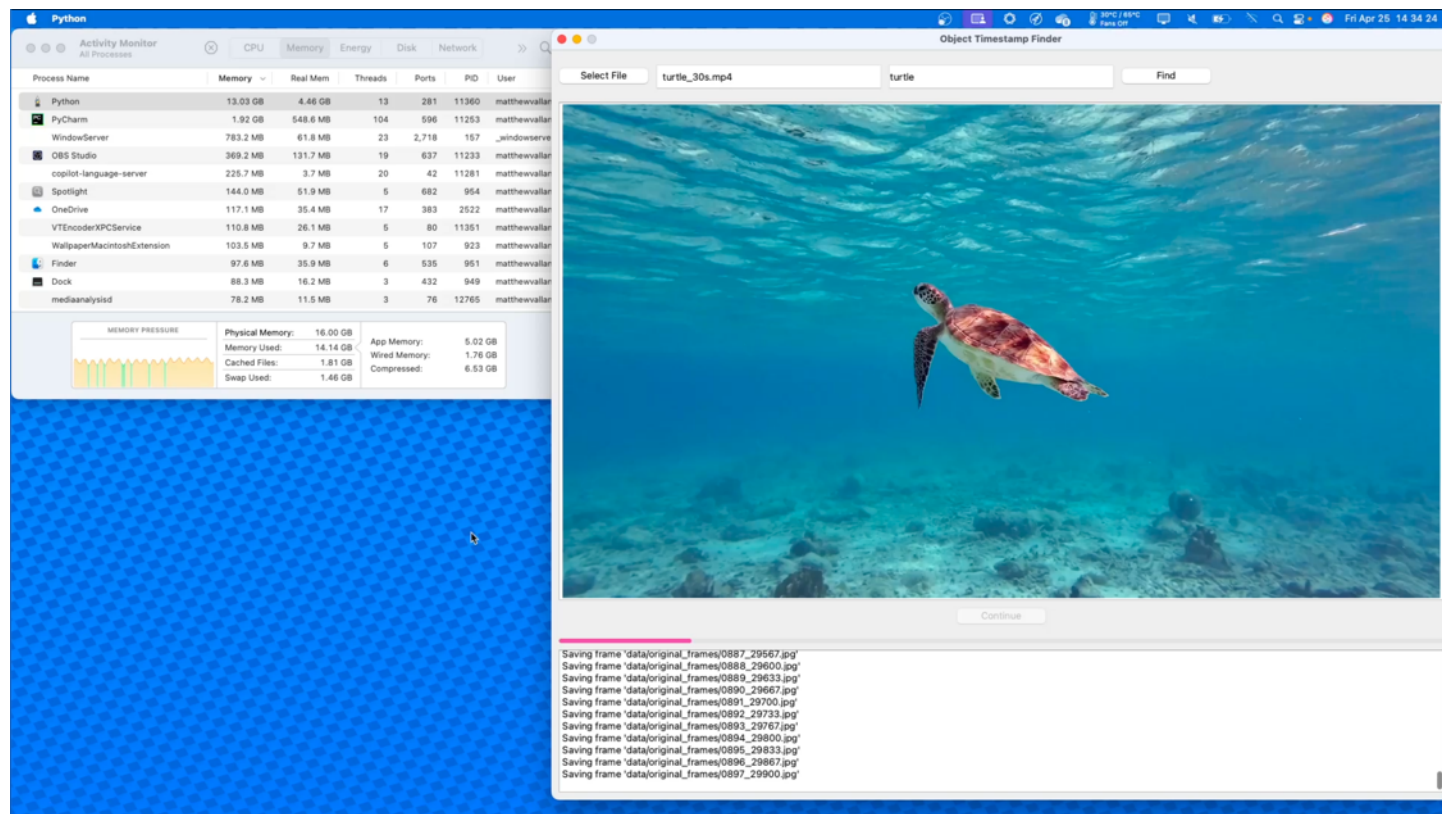
Matching Frame: 0569_18967.jpg

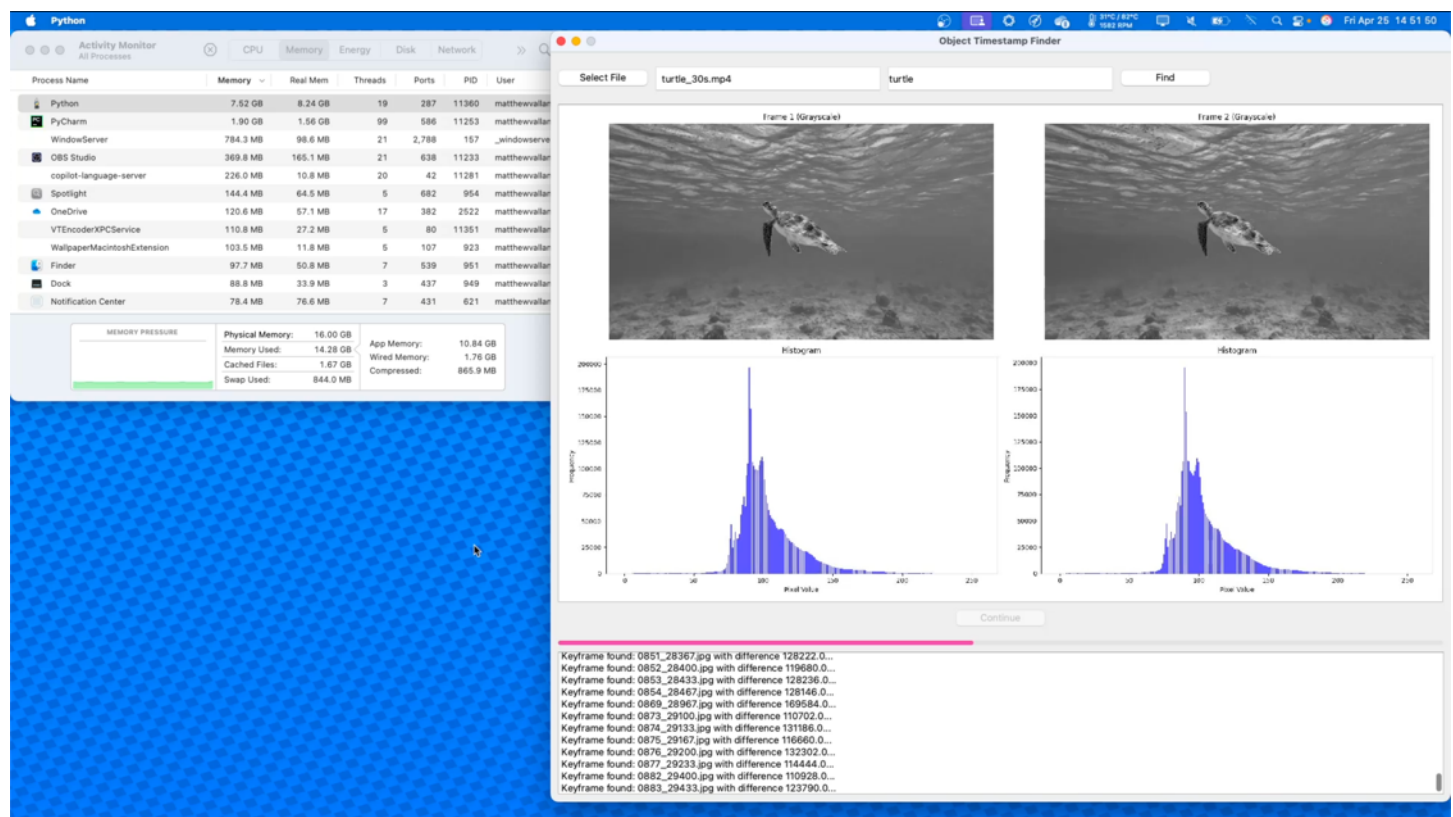
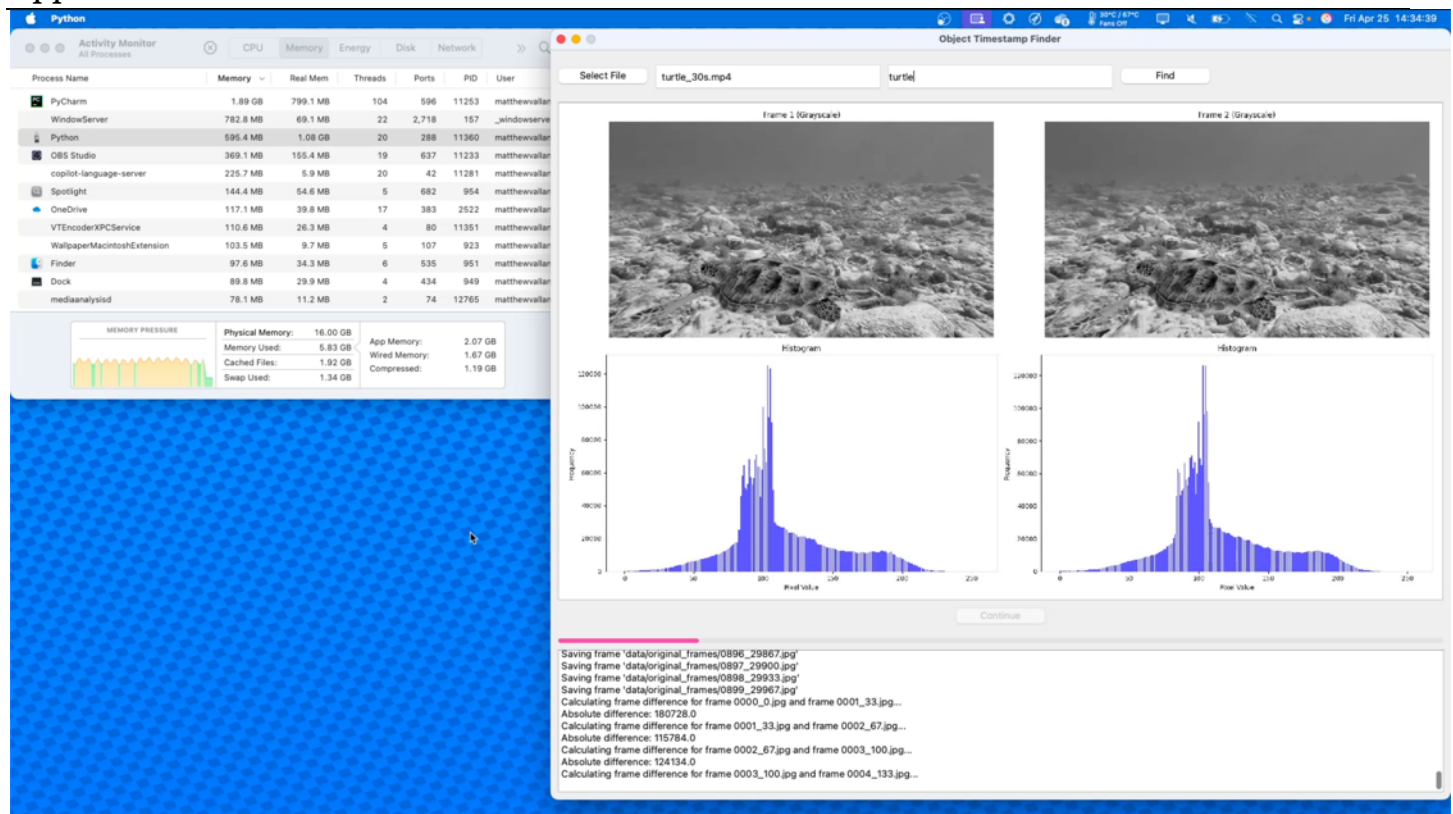
A photograph of a multi-lane highway with several cars and a large white semi-truck. The truck is in the center of the frame, moving away from the camera. The road is divided into two sections by a dashed line. The background is filled with trees and greenery, suggesting a rural or suburban setting.

Caption	Timestamp	Frame Number
The image shows a busy highway with multiple lanes of traffic. There are multiple cars and trucks on the road, including a white truck in the center of the image. The truck appears to be a large white semi-truck with a red stripe on the side. The road is divided into two sections, with the left side of the road being slightly blurred, while the right side is slightly out of focus. The background is filled with trees and greenery, indicating that the photo was taken during the day. The image is taken from a high angle, looking down on the highway.	00:18:966	0569

B.5 RAM usage

Note: Screenshots taken from various points of Appendix B.3.2 ‘turtle_30s.mp4’. A full run-through is available via the YouTube link. Screenshots have been taken at four places, towards the end of the frame extraction phase, at the start and towards the end of key frame calculations (and histogram generation), and at the report generation phase.





Python

Activity Monitor

All Processes

CPU

Memory

Energy

Disk

Network

Process Name	Memory	Real Mem	Threads	Ports	PID	User
Python (Not Responding)	19.29 GB	4.42 GB	24	538	11360	matthewvalla
PyCharm	1.88 GB	428.4 MB	89	545	11253	matthewvalla
WindowServer	783.2 MB	83.4 MB	22	2,817	157	_windowserve
OBS Studio	371.3 MB	123.5 MB	19	625	11233	matthewvalla
copilot-language-server	226.5 MB	3.6 MB	20	42	11281	matthewvalla
Spotlight	142.9 MB	29.6 MB	6	682	954	matthewvalla
OneDrive	116.7 MB	33.1 MB	17	381	2522	matthewvalla
VTEncoderXPCService	110.8 MB	17.2 MB	6	83	11351	matthewvalla
WallpaperMacintoshExtension	103.5 MB	9.5 MB	3	105	923	matthewvalla
Finder	97.6 MB	29.3 MB	7	536	951	matthewvalla
Dock	85.3 MB	29.4 MB	4	446	949	matthewvalla
Notification Center	77.0 MB	61.2 MB	7	433	621	matthewvalla

MEMORY PRESSURE

Physical Memory: 16.00 GB
Memory Used: 14.88 GB
Cached Files: 1.06 GB
Swap Used: 5.43 GB

App Memory: 5.50 GB
Wired Memory: 1.67 GB
Compressed: 6.88 GB

Object Timestamp Finder

Select File

turtle_30s.mp4

turtle

Find

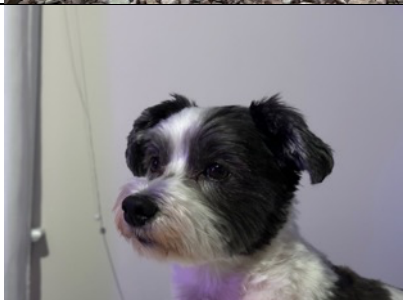


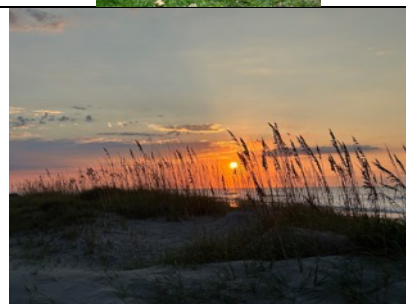
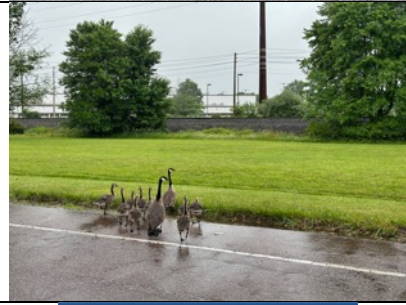
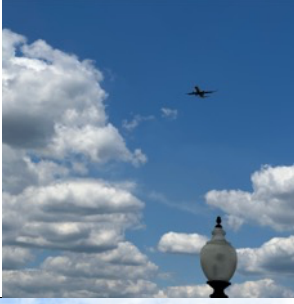
Continue

Processing paused, press continue to resume. Continuing in 2 seconds
Processing paused, press continue to resume. Continuing in 1 seconds
Processing resumed
Processing frame 188 of 188
Generated caption: The image is an underwater photograph of a sea turtle swimming in the ocean. The turtle is in the center of the image, with its body facing towards the right side of the frame. It has a brown shell with white spots on its head and back, and its flippers are spread out as it moves through the water. The water is a deep blue-green color and there are small rocks and coral reefs visible below the surface. The sun is shining brightly, creating a beautiful reflection on the water's surface.
Caption saved for frame
Search term found: turtle at 29.6 seconds
Processing resumed
All captions saved to CSV file.
Save location: /Users/matthewvallance/Downloads

Appendix C – Datasets, Videos and Images

C.1 Manually collected testing images

Image	Filename	Details (Reason or caption/Licence)
	black_bird.jpeg	Camouflage – The bird appears mixed into the plant pot behind it ©2024 Matthew Vallance
	cat_1.jpeg	Black cat on a stone background, camera from a high angle © 2024 Matthew Vallance
	cat_2.jpeg	Same image as first cat, but looking in a different direction © 2024 Matthew Vallance
	dog_1.jpeg	Wet dog on a rocky background, his legs blend into the background making his form tough to see © 2024 Matthew Vallance
	dog_2.jpeg	Obvious picture of a dog, against a white background © 2024 Matthew Vallance

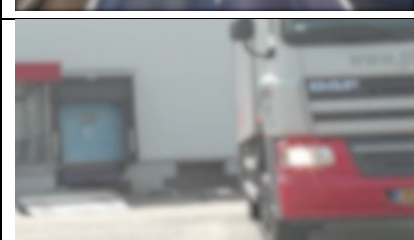
	ducks.jpeg	Two ducks on grass covered with leaves, these blend in easily © 2024 Matthew Vallance
	dunes.jpeg	No obvious objects, therefore what caption do we get? © 2022 Matthew Vallance
	geese_1.jpeg	Group of geese a similar colour to the road © 2024 Matthew Vallance
	geese_2.jpeg	Group of geese a similar colour to the road from further away © 2024 Matthew Vallance
	plane_lamp.jpeg	Plane and lamp against the sky, ratio of image is different © 2023 Matthew Vallance
	sp_athletics_water.jpg	Runners running through water, splashing in a stadium – to see impact of movement Pixabay Content Licence

	sp_high_street_people.jpg	People walking down a highstreet Pixabay Content Licence
	sp_highway_bridge.jpg	A highway bridge from far away – trying to spot specific vehicle colours would be tricky here Pixabay Content Licence
	sp_highway.jpg	A highway from slightly above – vehicles are closer in this image, so more specific details should be retrievable Pixabay Content Licence
	sp_horses_splashing.jpg	Horses running in water – this is to see the impact of water over parts of objects/animals Pixabay Content Licence
	sp_night_street_people_cars.jpg	A dark image of a street at night – to see how models react to darker images Pixabay Content Licence
	sp_people_walking_blur.jpg	Blurry crosswalk – to see how models react to blur/motion in images Pixabay Content Licence

	sp_red_car.jpg	A small red car driving through trees – should be an obvious caption Pixabay Content Licence
	sp_running_race.jpg	A busy race – how does the model count people in an image Pixabay Content Licence
	sp_trucks.jpg	Two trucks in the forefront of an image – this should be picked up as a ‘red truck’ and a ‘white truck’, very obvious. Pixabay Content Licence

C.2 Manually collected testing videos

Thumbnail	Filename	Details (Reason/Licence)
	bird_seed.mp4	Video contains birds flying in and out of the frame and pecking at the seed box Pixabay Content Licence
	flamingos.mp4	Video contains many flamingos, moving around for com Pixabay Content Licence
	highway_driving.mp4	Dashcam footage of driving down a highway, and pulling into a service station Pixabay Content Licence

	motorway_1.mp4	CCTV of highway pointed with the flow of traffic Pixabay Content Licence
	motorway_2.mp4	CCTV above highway pointed facing traffic Pixabay Content Licence
	snow_gull.mp4	A gull on a rusty pole, snow blurs view at points in the video Pixabay Content Licence
	town_square.mp4	Busy town square, with cars blocking the view at various points Pixabay Content Licence
	turtle.mp4	A single scene of a turtle swimming with the camera following it Pixabay Content Licence
	unfocussed_market.mp4	A blurred video of people walking around a market Pixabay Content Licence
	unfocussed_motorway.mp4	A blurred (towards top and bottom) CCTV video above the highway, with the traffic flowing towards the camera Pixabay Content Licence
	unfocussed_truck.mp4	A blurred video of a truck pulling out of a depot Pixabay Content Licence

C.3 Sample of TextCaps set

Note: This file contains over 100,000 lines of text, so only the first 10 **valid** items of the JSON data are included here, with only the data used within this project included as table headings. The full dataset is available at: <https://textvqa.org/textcaps/download/>

Image	Reference Captions
 https://c7.staticflickr.com/4/3905/15210925691_e685eeabda_z.jpg	Five Listerine Zero mouthwash bottles on a store shelf Many bottles of Listerine Zero are placed together. Listerine Zero mouthwash is less intense with zero alcohol. Several bottles of Listerine Zero sit on display on a shelf Listerine zero bottles which is less intense and zero alcohol.
 https://c6.staticflickr.com/9/8500/8295861631_2c202c18c8_z.jpg	The time is now 10:22 according to this clock. A large clock by Harris and Thompson displays a time of 10:22. Large roman numeral clock by Angel Square London. Clock with roman numeral numbers which say "Angel Square" on the front. A clock says Angel Square London on its face.
 https://c1.staticflickr.com/2/1097/1206594541_0fa8b6dc15_z.jpg	Lloyd Budd's email service is powered by Google. A Gmail screen shot welcoming Lloyd Budd asking for Username and Password. A Gmail account is connected to something called Lloyd Budd. a mail page with Lloyd Bud on it A fire fox login page to lloy budd gmail.
 https://c2.staticflickr.com/3/2877/10037296734_0b7bcca795_z.jpg	The laptop can support HDMI, USB, and CDs. A closeup of the side of a laptop with the word HDMI the side of a laptop has an HDMI port marked a somewhat blurry picture of what seems to be a laptop showing HDMI outlet on a table. The side of a laptop is shown and the HDMI port is clearly labeled and visible.
 https://c5.staticflickr.com/2/1165/526319721_0e33f8c855_z.jpg	The parking meter takes quarters and is expired. A parking meter is a two hour time limit is showing that the time has expired. a parking metter displaying that time has expired The parking meter warns that time has now expired. A parking meter with a two hour time limit reads time expired.

 https://c3.staticflickr.com/8/7315/8721004582_9ed63c90cb_z.jpg	The bookshelf consists of about 20 different types of books. Many books are on a shelf including a green one that is titled The Complete Grunfeld. A shelf of books contains the title Chess for Beginners. A row of books on a shelf and one of them says The Complete Grunfeld. Multiple books on a book shelf, one of which is The complete GrunFeld.
 https://c7.staticflickr.com/5/4023/4268554966_02919c1f8d_z.jpg	The black computer monitor screen is made by LG. A monitor with the letters LG on the bottom. A black LG monitor with a white background. A black monitor says LG on the front and is on a white background. An LG flat screen television with SOURCE, MENU, AUTOSET, and LIGHT buttons
 https://c6.staticflickr.com/5/4008/4614836112_6813692c03_z.jpg	A sports team from England is holding up their winning trophy. screen showing several men wearing blue shirts with england on front and one of them holding a trophy A sports team gathering, all players with England on their shirts. Men wearing black shirts saying England are gathered together. an England shirt on the person that is outside
 https://c2.staticflickr.com/7/6237/6300940083_8ba7783f60_z.jpg	A framed poster of the cast of the movie Top Gun. wall display with multiple photos and the top gun words and emblem at bottom a large picture with the title of Top Gun a picture that has Top Gun written at the bottom Poster framed on a wall for the movie Top Gun.
 https://c2.staticflickr.com/1/210/496424889_a4878683f0_z.jpg	A bottle of Alhambra lager sits next to the cup it was poured into on a table in a restaurant. A bottle of Alhambra lager next to a full glass of beer on a table. A restaurant table holds a Alhambra bottle with a glass full next to it. a bottle of alhambra pemium lager standing next to a glass of it on a counter a glass and bottle of Alhambra Premium Lager



Appendix D – Declaration of AI use

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – NO
- To create an outline of the topic – NO
- To explain concepts – YES (**NOT used within the report, only for developing a personal understanding of subject matter. Read and not copied directly**)
- To support my use of language – NO
- To summarize articles/resources – NO
- In other ways, as described below:
 - To source academic papers from open-access resources (via Microsoft Copilot)